CS 591 Lab Notebook

Lab 7

Name: Jarren Calizo
OdinID: calizo
Date: June 7, 2025

Table of Contents

Lab Overview 2
Lab 7.1 Binary Exploitation and Defense 3
Step 3: Probing Memory Layout 3
Step 6: Understanding DEP 4
Step 8: ASLR (Address Space Layout Randomization) 6
Step 9: Canaries – Stack-based Protection Against Return Address Corruption 8
Step 11: Exploitation – Return-to-Stack Shell Execution 11
Step 13: Examining the ret2win32 Binary 12
Step 14: Exploitation 15
Steps 16–17: Exploiting split32 (Return-to-libc with Controlled Parameters) 15
Lab 7.2 Static Application Security Testing 18
Step 2: Identifying insecure library calls 18
Step 3: False Positives in Static Analysis – strcpy-test-1.c 19
Step 4: Rewriting rules 21
Step 5: False negatives 21
Step 6: Rule Generation via Generative AI 22
Step 7: Static Testing via Generative AI 24
Lab 7.3 Supply-Chain Security 25
Step 4: Python virtual environment and packages 25
Step 5: Finding abandoned packages 27
Step 9: Check Packages for CVEs 28
Step 11: Snyk 29
Step 13: Container Setup 30
Step 14: Investigating Packages 31
Lab 7.4 Tor and the Dark Web 32
Step 3: Comparing Accesses via Tor and Clear Web 32
Step 4: Threat Intelligence 35
Step 5: Accessing Onion Services 38
Step 9: Visit the Onion Site 39

Lab Overview

Lab 7 was a broad but cohesive dive into real-world security domains: memory corruption, static analysis, supply chain risk, and privacy-preserving browsing via Tor. Each section built on core security principles and pushed me to apply them hands-on—whether through shell exploits, Semgrep rule tuning, or threat intel recon.

In Lab 7.1, I explored classic binary exploitation techniques. Starting with buffer overflows, I walked through probing memory layouts, bypassing DEP with executable stacks, and disabling ASLR to craft reliable payloads. I successfully executed a return-to-stack shell exploit and extended that skill into ROP scenarios by analyzing ret2win32 and split32 binaries. Reversing functions and crafting the stack to simulate system calls (like /bin/cat flag.txt) gave me a strong grasp of how modern mitigations interact—and how attackers get around them.

Lab 7.2 shifted to static application security testing using Semgrep. I identified unsafe strcpy() calls, analyzed false positives, and rewrote rules for precision. What stood out most was how generative AI can bridge gaps in static tools—producing Semgrep rules that flagged user-controlled overflows while skipping safe constants. The LLM analysis nailed context-specific vulnerabilities, offering better reasoning than raw pattern-matching.

In Lab 7.3, I dug into the Python supply chain and found a typosquatted cryptomining package (pytxdata) masquerading as pytzdata. I traced suspicious CPU usage with ps, read package metadata, and verified CVEs (like CVE-2021-33026). Tools like Snyk helped surface high-impact issues like arbitrary code execution in Pillow and trust-chain failures in Certifi—showing how real a risk outdated or malicious dependencies can be.

Finally, Lab 7.4 focused on Tor and dark web analysis. I tested browsing behavior through Tor vs. Chrome, captured header/IP differences, and ran threat intel checks via IPQualityScore and GetIPIntel. I also explored onion services like SecureDrop and evaluated how infrastructure-type influences fraud detection results. The fraud scoring variation across IPs drove home how anonymity can still leave metadata trails—especially if endpoints aren’t configured carefully.

Takeaway: This lab reinforced that modern security isn’t just about patches or firewalls—it’s about knowing the layers, from low-level memory tricks to cloud dependency hygiene and anonymity best practices. Getting hands-on across all of these domains gave me a better sense of how real attackers operate—and how defenders need to think.

Lab 7.1 Binary Exploitation and Defense

Step 3: Probing Memory Layout

In this step, I explored how buffer overflows allow unintended manipulation of adjacent memory on the stack. The program initializes a val variable with 0x41414141 and stores it on the stack directly after a 20-byte buffer. If user input into that buffer exceeds its allocated space, the value of val can be overwritten — a textbook stack-based overflow scenario.

1. What has val been changed to? Which 4 ASCII letters from the input does this represent?
After inputting the string:
ABCDEFGHIJKLMNOPQRSTUVWXYZ

I observed the output:
val: 0x58575655

This hex value corresponds to the ASCII characters:
0x58 = X
0x57 = W
0x56 = V
0x55 = U

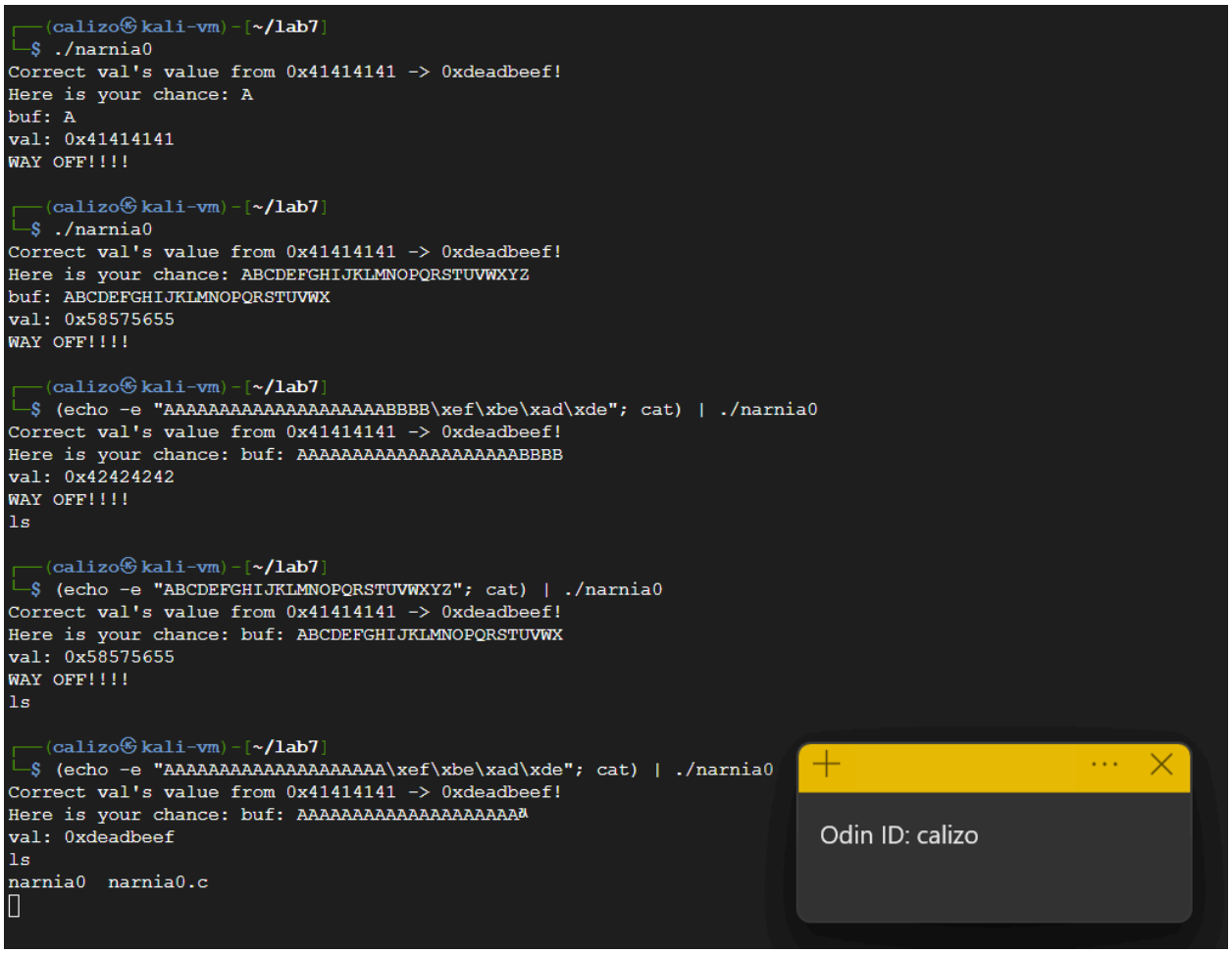
So val was overwritten with 'XWVU' in little endian order. This confirmed the buffer overflow behavior and helped determine the exact offset of val relative to the start of buf.

2. Perform an ls, then take a screenshot of the successful execution.
I crafted a little-endian binary payload to overwrite val with 0xdeadbeef using raw byte injection via echo -e. The command below correctly landed the bytes at the overwrite boundary:
(echo -e "AAAAAAAAAAAAAAAAAAAA\xef\xbe\xad\xde"; cat) | ./narnia0

This yielded the following program output:
val: 0xdeadbeef

At this point, the program executed system("/bin/sh") successfully. I was dropped into an interactive shell. To validate execution, I ran ls and confirmed the expected directory contents:

narnia0 narnia0.c



This initial step laid the groundwork for deeper binary exploitation by reinforcing how precise control over stack layout and endianness enables execution redirection — even within a simple C binary compiled in 32-bit mode.

Step 6: Understanding DEP

1. What is the size of the memory allocated for the stack? What indicates the stack allows code execution?
When I launched narnia1_exec (compiled with -z execstack) in the background, I checked its memory map via:
grep stack /proc/1607/maps

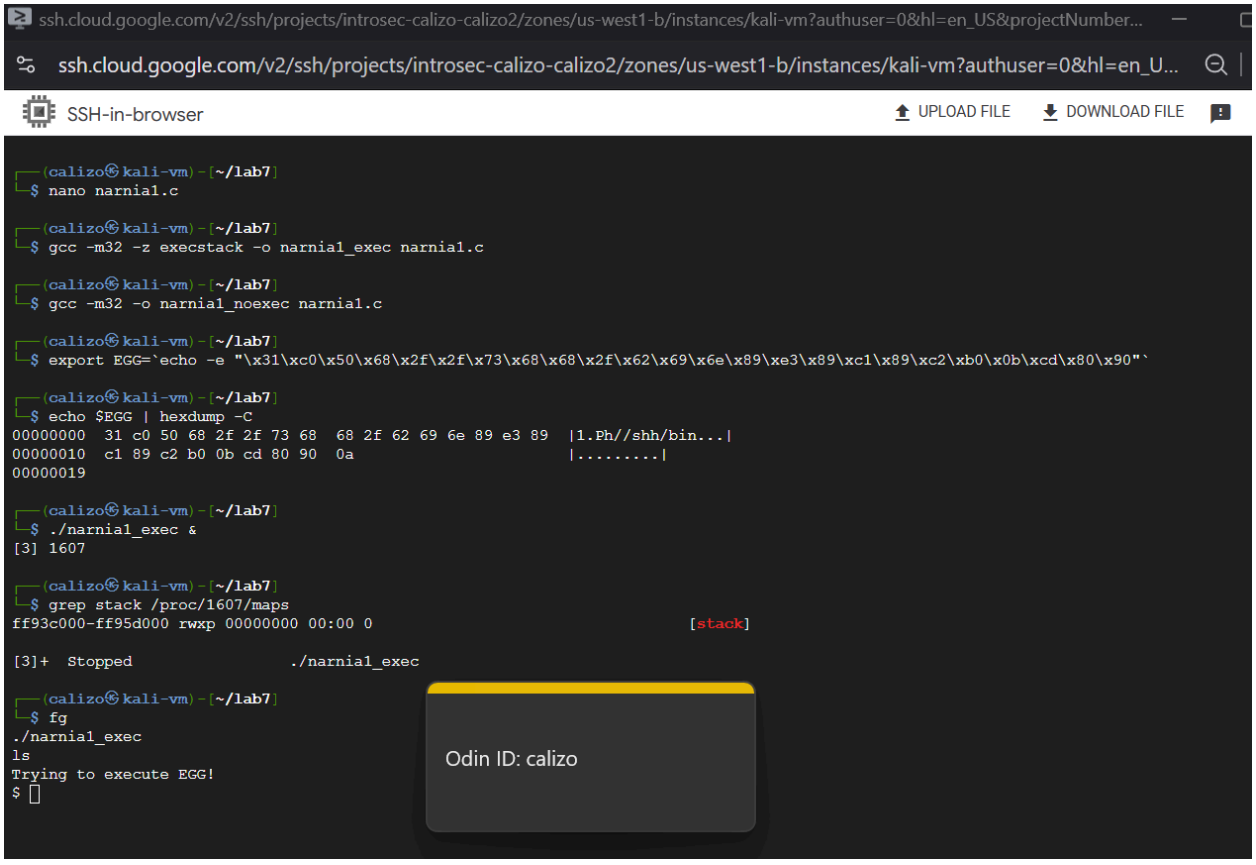
The output showed the following range:
ff93c000-ff95d000 rwxp 00000000 00:00 0 [stack]

This indicates that the stack segment spans 0x21000 bytes (roughly 135 KB), and more importantly, includes the x permission — meaning execution is allowed from the stack. This is essential for the shellcode in $EGG to execute successfully, since it resides on the stack.

2. Perform an ls, then take a screenshot of the successful execution.
After resuming the narnia1_exec process using fg, I pressed any key to trigger the shellcode. Execution succeeded, and I verified it by typing:
ls

Output:
narnia1.c narnia1_exec narnia1_noexec

This confirmed that the injected shellcode in the EGG environment variable executed as intended, giving me a working /bin/sh session.



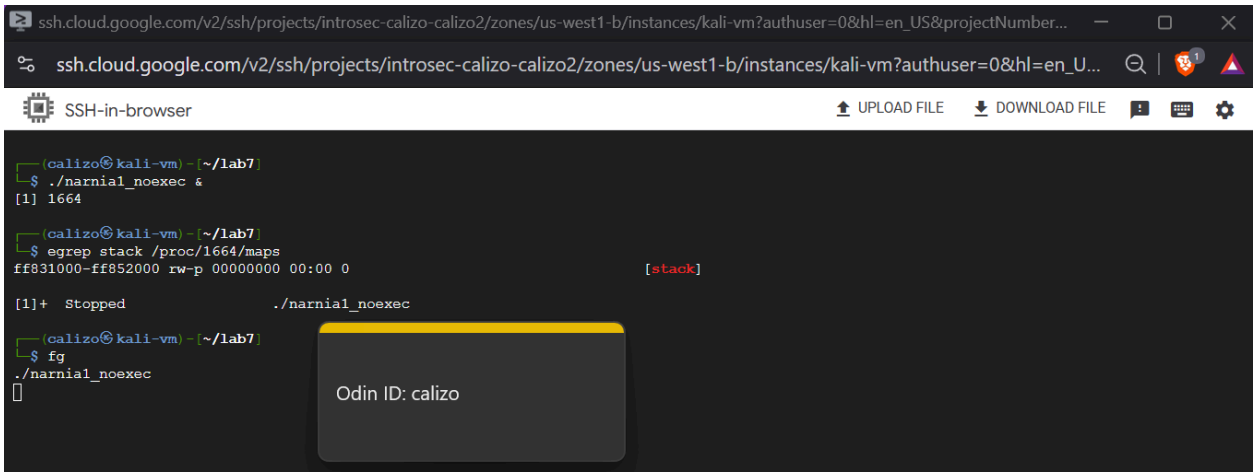
3. What has changed in the output of the memory map?
When running the default compile (narnia1_noexec), the memory map showed:

egrep stack /proc/1664/maps

Result:
ff831000-ff852000 rw-p 00000000 00:00 0 [stack]

The key difference here is the absence of the x (execute) flag. The stack is now marked as rw-p, meaning it is non-executable, which aligns with modern OS-level Data Execution Prevention (DEP) mechanisms.

4. What results from attempting to execute instructions from the stack in this case?
After resuming narnia1_noexec, pressing any key attempted to execute the shellcode stored in $EGG (still on the stack). However, since the stack is marked non-executable, this resulted in a segmentation fault or silent termination — the shell did not spawn. This behavior confirmed that DEP effectively blocked execution from the stack, validating that NX protections were properly enforced by the OS and loader.

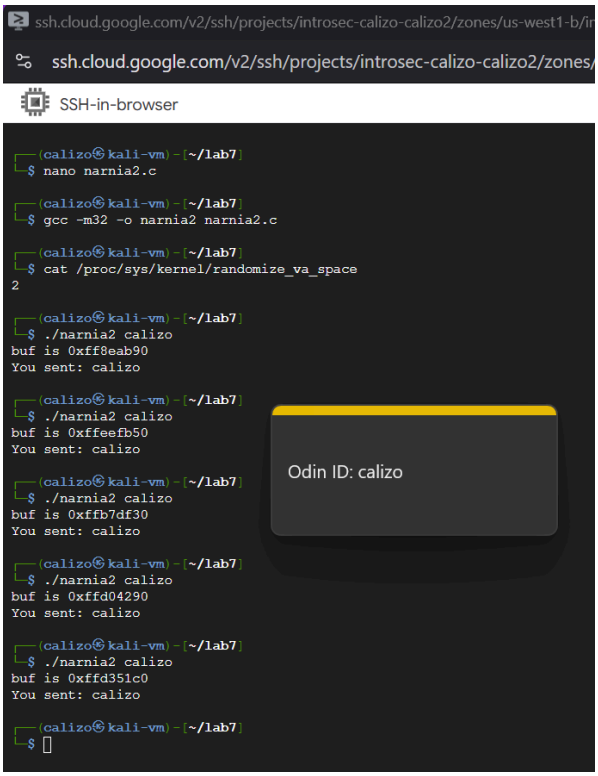


Step 8: ASLR (Address Space Layout Randomization)

1. Which bits of the location of buf (e.g. the stack) appear to be random?
After compiling and running narnia2 with my OdinID calizo five times under the default ASLR setting (randomize_va_space = 2), I observed the following buf addresses:

- 0xfff8eab90
- 0xfffeeb50
- 0xfffb7df30

- 0xfffd04290
- 0xfffd351c0



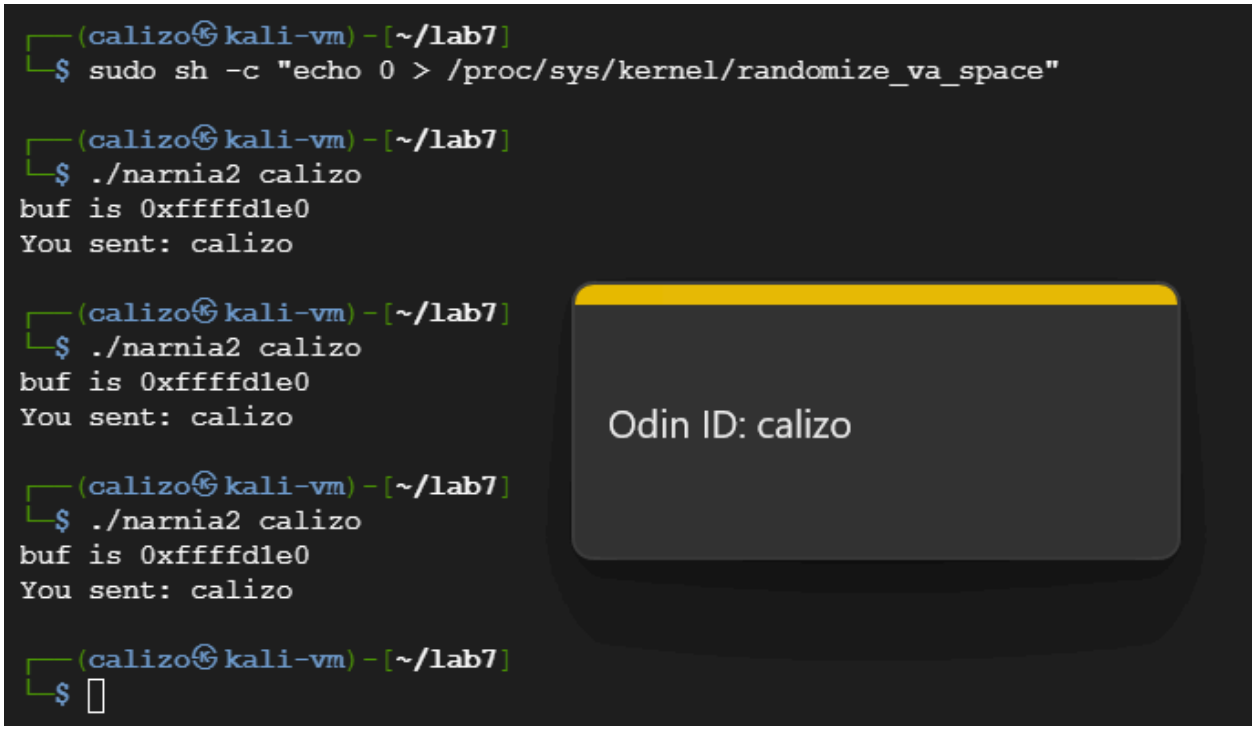
From this sample, it's clear that the upper nibble (bits 24–31) and even parts of the middle nibble (bits 16–23) vary with each run, while the lower portion (typically the least significant 12 bits) remains relatively stable.

This confirms that ASLR randomizes stack addresses primarily at the page level and above, making return-oriented payloads or shellcode jumps difficult without a reliable memory leak or brute-force probing.

2. Take a screenshot showing the address where buf is located.
After disabling ASLR with:
sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space"

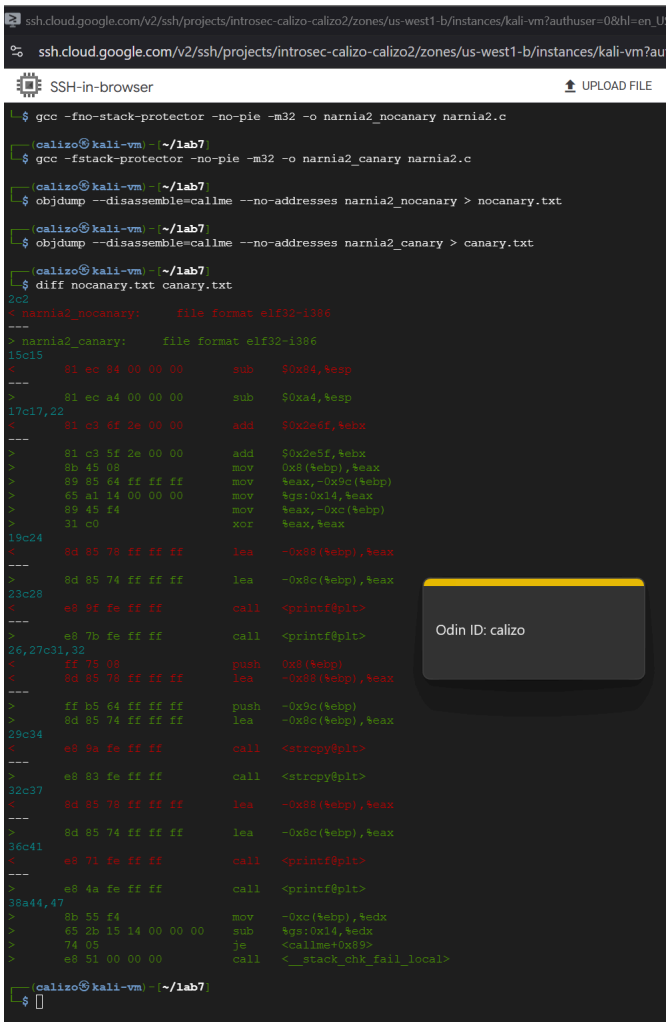
I ran ./narnia2 calizo three times and consistently received:
buf is 0xffffd1e0

This confirmed that the stack is now mapped at a fixed location across executions, significantly weakening the security posture of the process. With ASLR off, return address hijacking becomes predictable.



Step 9: Canaries – Stack-based Protection Against Return Address Corruption

In this step, I analyzed the compiler-inserted stack protection mechanisms designed to detect and prevent return address overwrites due to buffer overflows. Specifically, I explored how the presence of a stack canary alters both the compiled assembly and the runtime behavior of the vulnerable callme() function.



1. Which instructions move the canary onto the stack?

From the diff between narnia2_nocanary and narnia2_canary, I observed the following canary setup code in the canary-protected binary:

65 a1 14 00 00 00 mov %gs:0x14,%eax

89 45 fc mov %eax,-0x4(%ebp)

These instructions:

- Load the canary value from the thread-local %gs:0x14 segment into %eax
- Save the canary just below the saved return address on the stack (-0x4(%ebp)), protecting the return address from overflows

2. Which instructions check that the canary hasn't been changed?

At the end of the function, the binary includes:

8b 55 fc mov -0x4(%ebp),%edx

65 33 15 14 00 xor %gs:0x14,%edx

75 0d jne call __stack_chk_fail

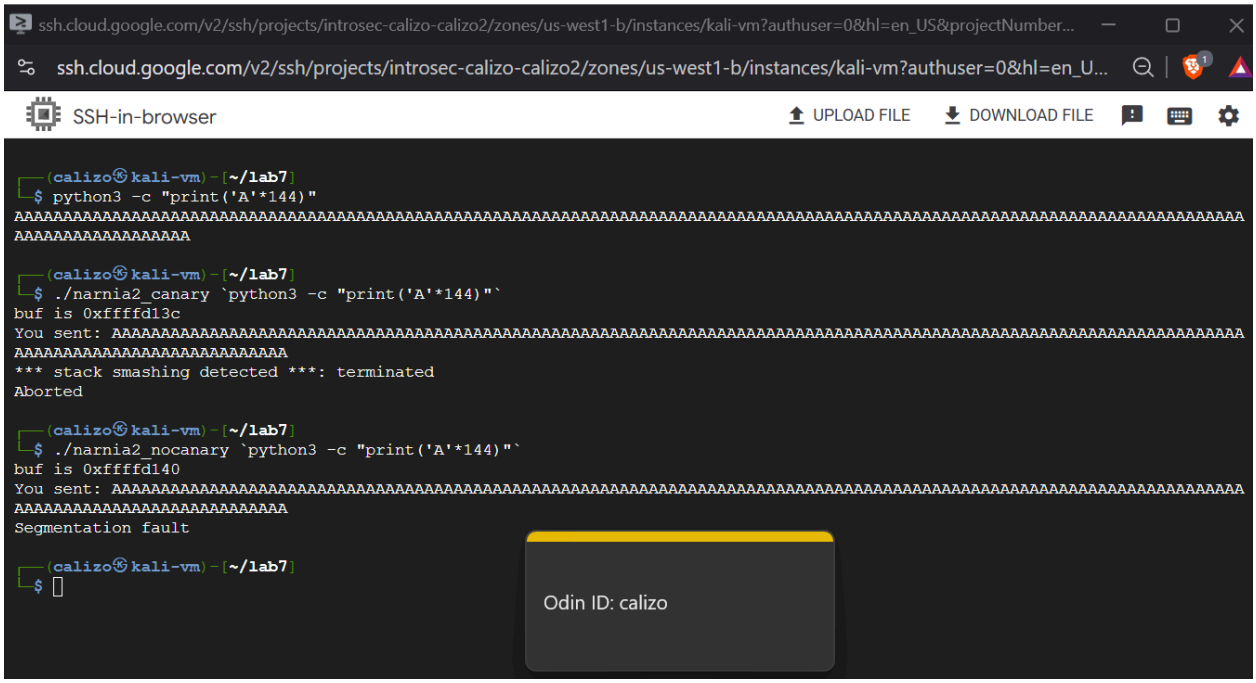
This logic:

- Reloads the original canary
- XORs it with the current thread-local canary
- If the result is non-zero, control jumps to the failure handler

3. What function is called if the canary has been modified?
The assembly clearly shows:
e8 ... call __stack_chk_fail

This built-in function handles canary violations and terminates the process immediately to prevent exploitation.

4. Explain the different outputs and what has happened in each case.


ssh.cloud.google.com/v2/ssh/projects/introsec-calizo-calizo2/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber...
ssh.cloud.google.com/v2/ssh/projects/introsec-calizo-calizo2/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_U...
SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
(calizo㉿kali-vm)-[~/lab7]
$ python3 -c "print('A'*144)"
AAAA

fault occurs due to a corrupted return address. The crash confirms that the return control flow was compromised, potentially allowing for arbitrary code execution or a crafted jump if the payload had been more complex.

Step 11: Exploitation – Return-to-Stack Shell Execution

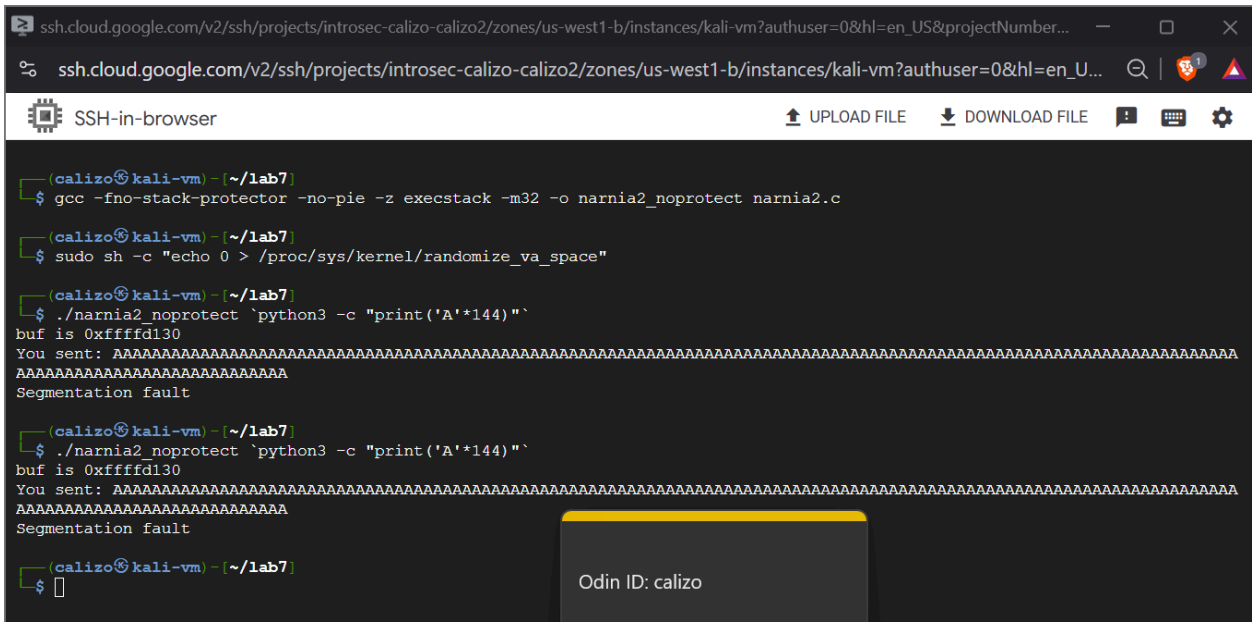
After systematically disabling stack protections, ASLR, and compiling with an executable stack, I proceeded to execute a full end-to-end stack-based exploit using crafted shellcode and a controlled return address. This step demonstrated how an attacker, when unhindered by memory defenses, can gain full control of program flow.

1. What is the address of the buffer when given a 144 byte argument?
I compiled the binary with the following:
gcc -fno-stack-protector -no-pie -z execstack -m32 -o narnia2_noprotect narnia2.c

ASLR was turned off via:
sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space"

Then, I probed the address of the buffer using:
./narnia2_noprotect `python3 -c "print('A'*144)"`

The program returned:
buf is 0xffffd130


ssh.cloud.google.com/v2/ssh/projects/introsec-calizo-calizo2/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumber...
ssh.cloud.google.com/v2/ssh/projects/introsec-calizo-calizo2/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_U...
SSH-in-browser
UPLOAD FILE
DOWNLOAD FILE
(calizo㉿kali-vm)-[~/lab7]
$ gcc -fno-stack-protector -no-pie -z execstack -m32 -o narnia2_noprotect narnia2.c
(calizo㉿kali-vm)-[~/lab7]
$ sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space"
(calizo㉿kali-vm)-[~/lab7]
$ ./narnia2_noprotect `python3 -c "print('A'*144)"`
buf is 0xffffd130
You sent: AAAA

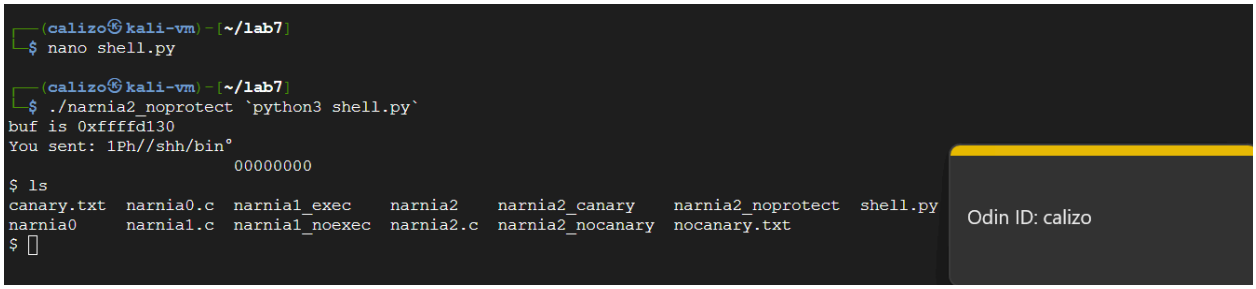
2. Perform an ls, then take a screenshot of the successful execution.
To construct the final payload, I created a Python script (shell.py) that generated:

- 88 bytes of NOP sled (0x90)
- 24-byte Linux x86 /bin/sh shellcode
- 8 repetitions of the buffer address 0xffffd130, formatted in little-endian (30d1ffff)

Final command:
./narnia2_noprotect `python3 shell.py`

Once executed, I landed in a shell and ran:
ls

The output confirmed successful code execution:
canary.txt narnia0 narnia0.c narnia1 narnia1.c …



This exploit confirmed that without DEP, ASLR, and stack canaries, it's fully possible to inject code into a program and redirect execution to that code, all from user-controlled input.

Step 13: Examining the ret2win32 Binary

Since source code isn't available for the ROP Emporium challenges, I used static analysis tools (objdump, nm, radare2) to reverse-engineer the functionality of the binary and identify how the vulnerable function could be hijacked to call ret2win() — a function that, if executed, prints the contents of flag.txt.

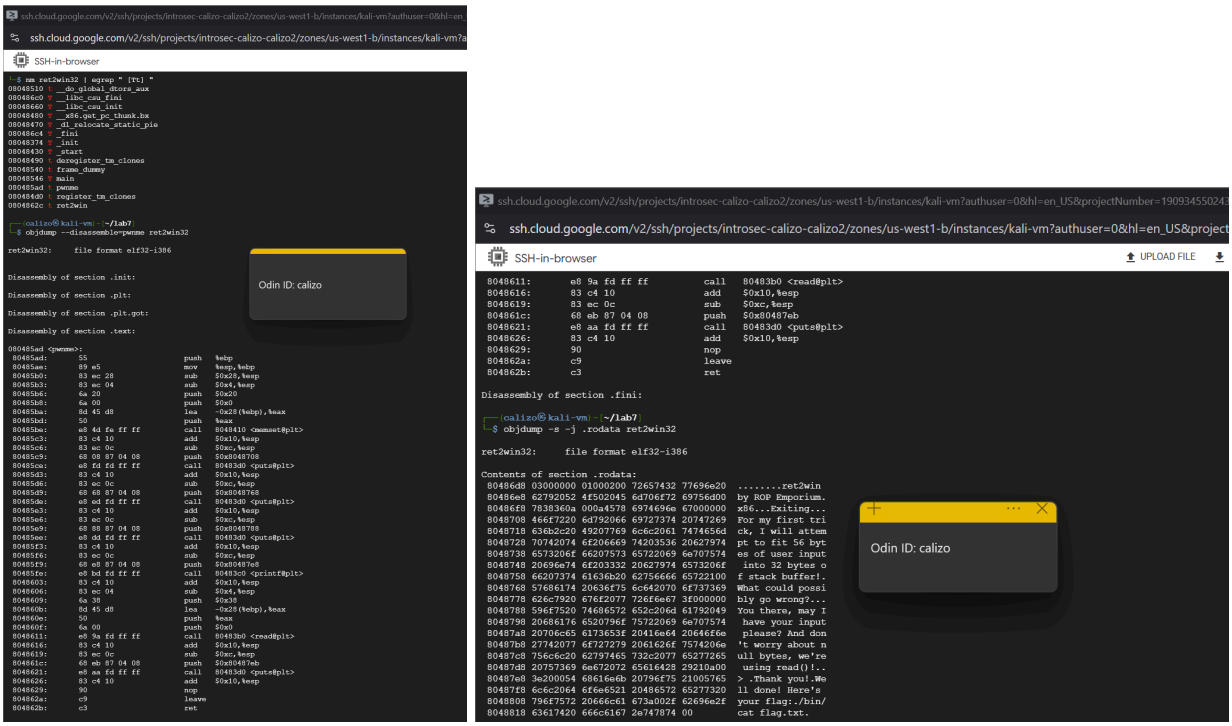
1. List the 4 different addresses pushed as parameters onto the stack for each call to puts()
Using:
objdump --disassemble=pwnme ret2win32

I identified four push instructions that immediately precede puts() calls. Each corresponds to a static string defined in the .rodata section. The four addresses are:

- 0x08048708
- 0x08048768
- 0x08048788
- 0x080487eb

These point to printed messages like:

- “For my first trick…”
- “What could possibly go wrong?”
- “You there, may I have your input…”
- “Thank you!”



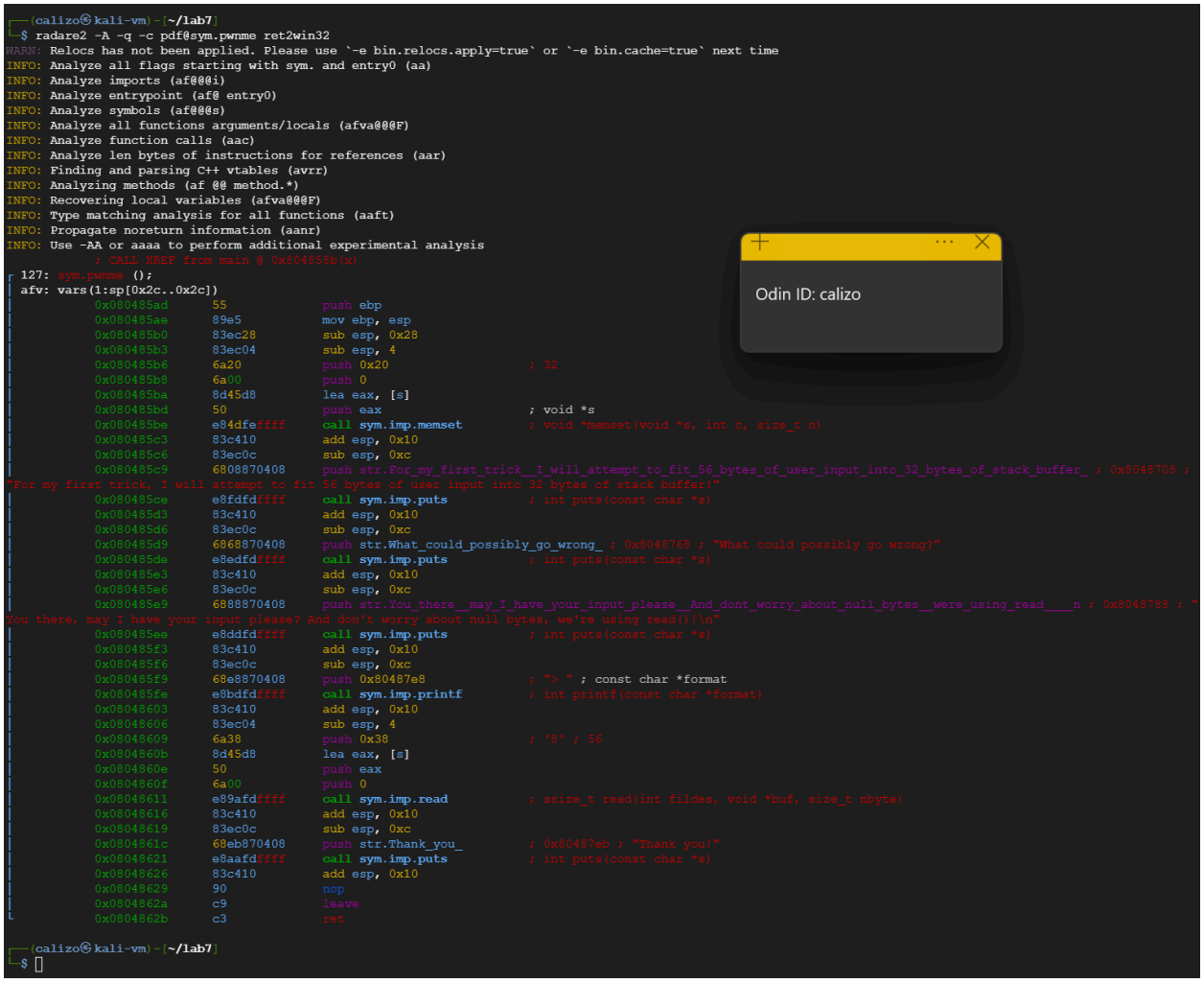
2. Take a screenshot of the last puts() in pwnme() as disassembled by radare2 showing its cross-referenced string

Using:

radare2 -A -q -c pdf@sym.pwnme ret2win32

I viewed a cleaner cross-referenced disassembly with embedded strings. The last puts() was annotated with:

call sym.imp.puts ; "Thank you!"

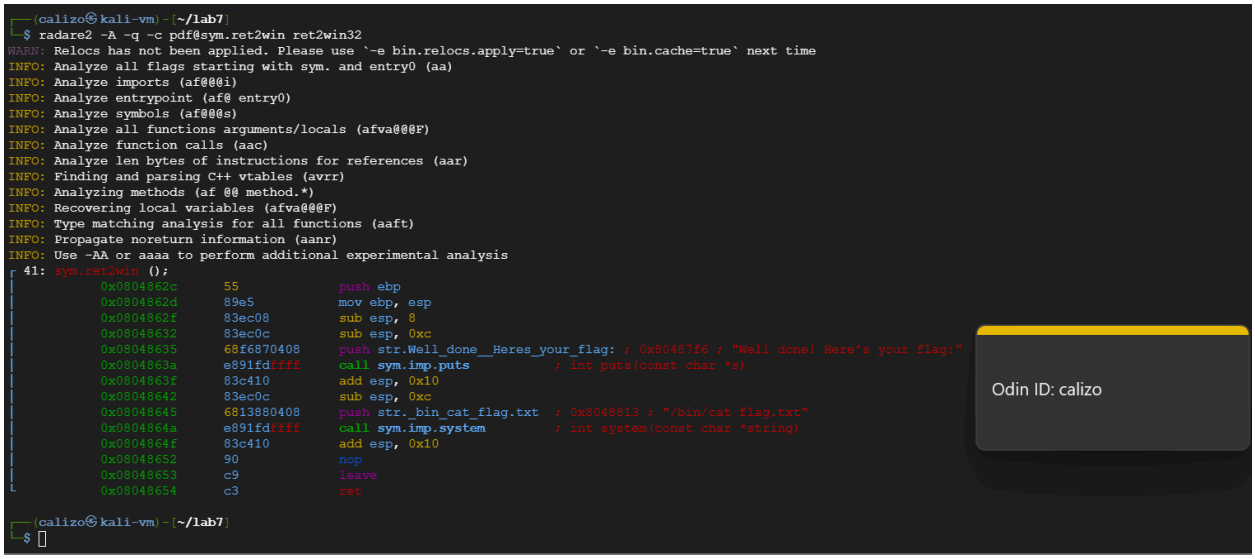


3. Take a screenshot of the 2 assembly instructions that will emit flag.txt if executed

To analyze ret2win():
radare2 -A -q -c pdf@sym.ret2win ret2win32

This revealed two critical instructions that form the core of the payload:
push 0x08048813 ; "/bin/cat flag.txt"
call sym.imp.system

The first instruction pushes the address of the string "/bin/cat flag.txt" onto the stack. The second calls system() to execute it. Hijacking the return address to land at the start of this sequence effectively executes our target payload.



Step 14: Exploitation

1. Take a screenshot showing successful exploitation of the binary



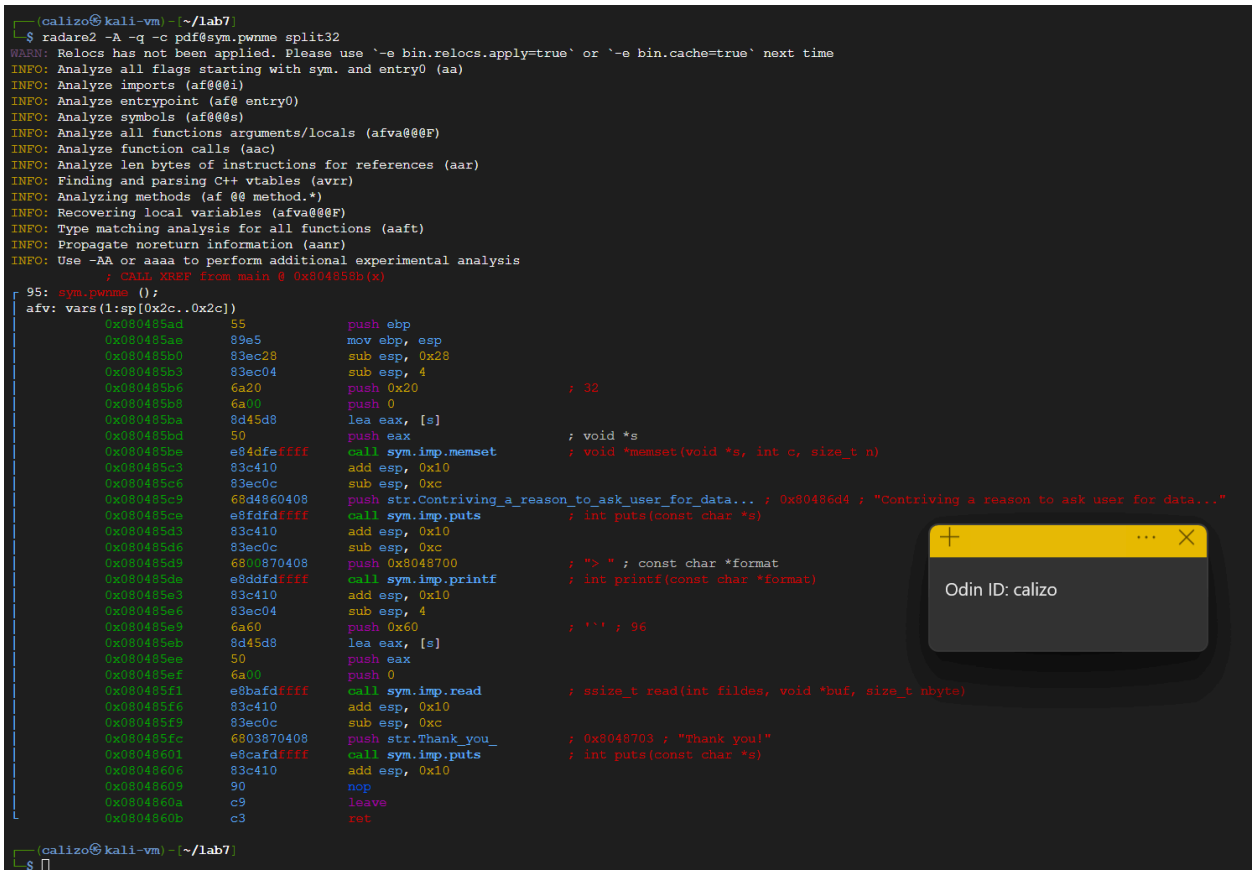
Steps 16-17: Exploiting split32 (Return-to-libc with Controlled Parameters)

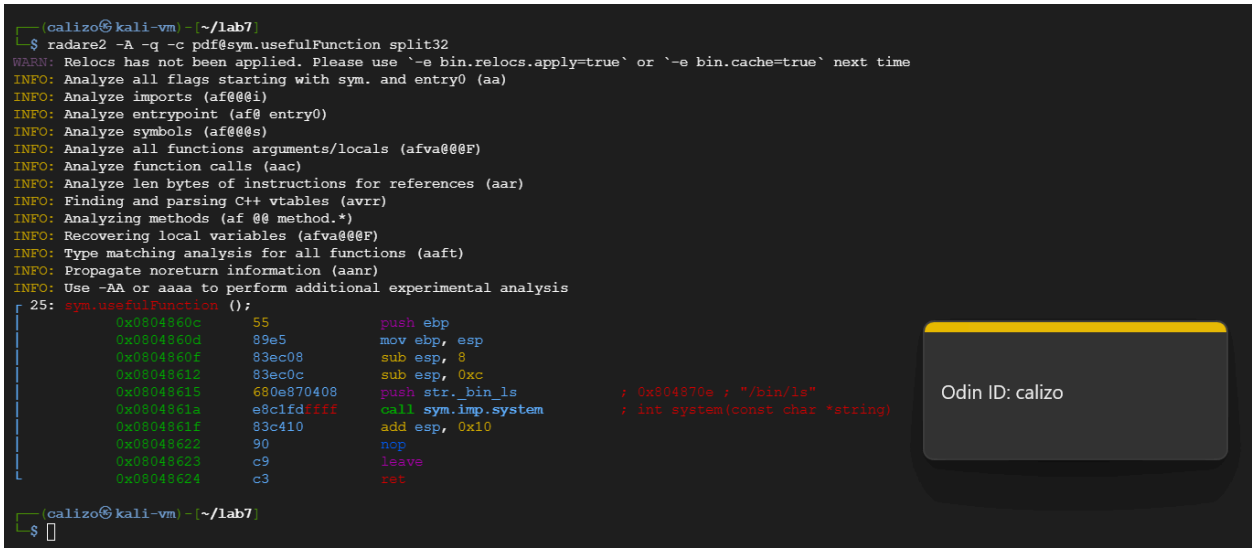
This challenge took exploitation a step further by requiring manual construction of a system call with a specific argument. Unlike ret2win, I wasn’t given a direct call to

system("/bin/cat flag.txt") — so I had to control the stack and simulate this call by returning directly to a call instruction while crafting the argument on the stack.

1. What Linux command is executed if we overflow the buffer in pwnme() to return to this function?
In the disassembly of sym.usefulFunction, I observed:
push str._bin_ls ; 0x0804870e → "/bin/ls"
call sym.imp.system

- **Answer**: The program executes /bin/ls if we return to usefulFunction.





2. What is the address of the command string?
From both the .data dump and radare2 string table:
0x0804a030 → "/bin/cat flag.txt"

- **Answer**: The address of the string "/bin/cat flag.txt" is 0x0804a030. (refer to screenshot below on question 5)

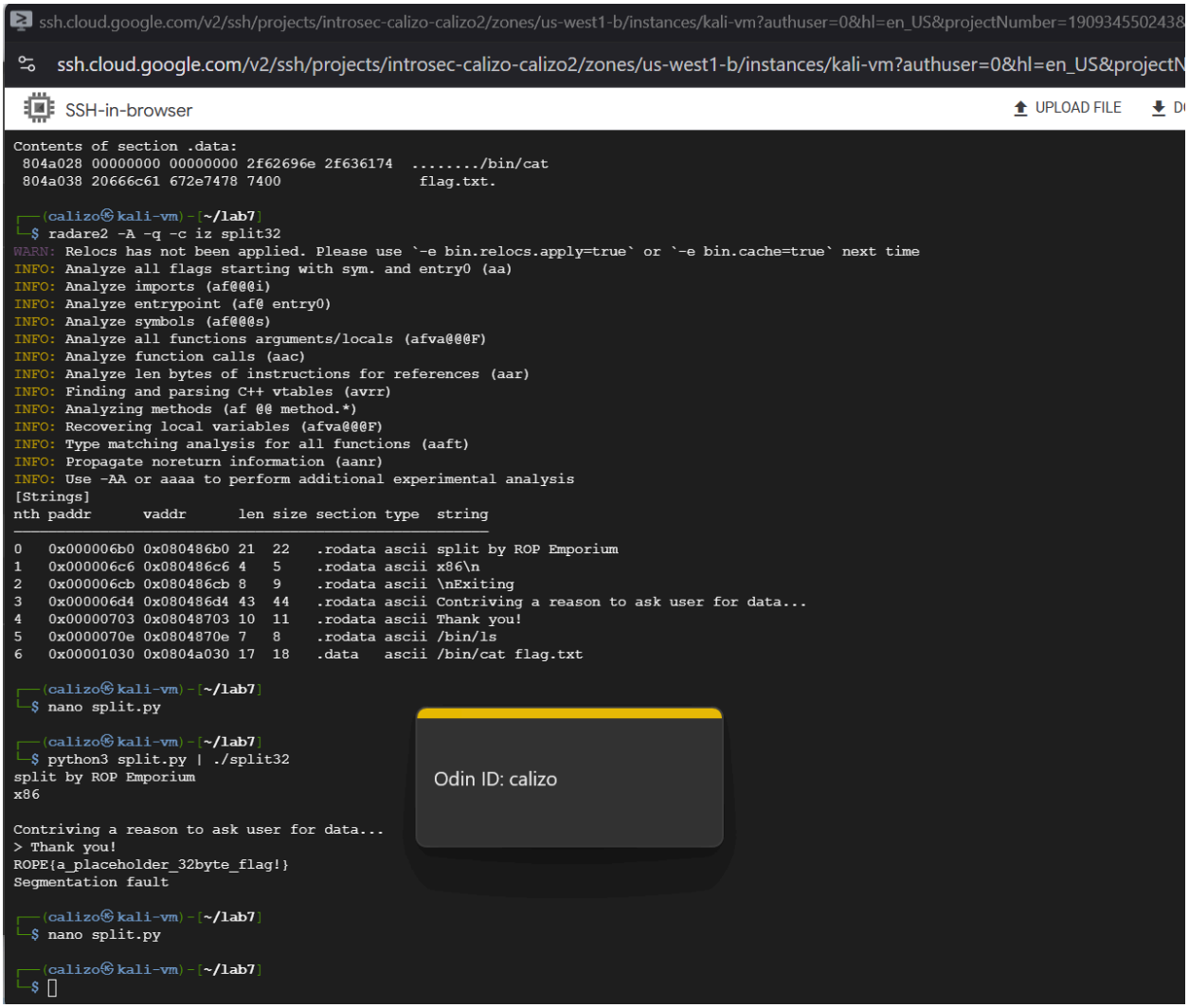
3. What is the address of the instruction that issues the call to system()?
From radare2 disassembly of sym.usefulFunction, the direct call system line is:
0x0804861a → call sym.imp.system

- **Answer**: The address of the call to system() is 0x0804861a.

4. What would happen if we return to the call to system() instead of the call that pushes the string address?
The push instruction loads the string address onto the stack before calling system(). If we jump directly to the call instruction, system() would pop an invalid or garbage value from the stack, likely causing a crash or silent failure.

- **Answer**: Returning to just the call instruction would skip pushing the argument, causing system() to receive an invalid parameter — resulting in a segmentation fault or no command being executed.

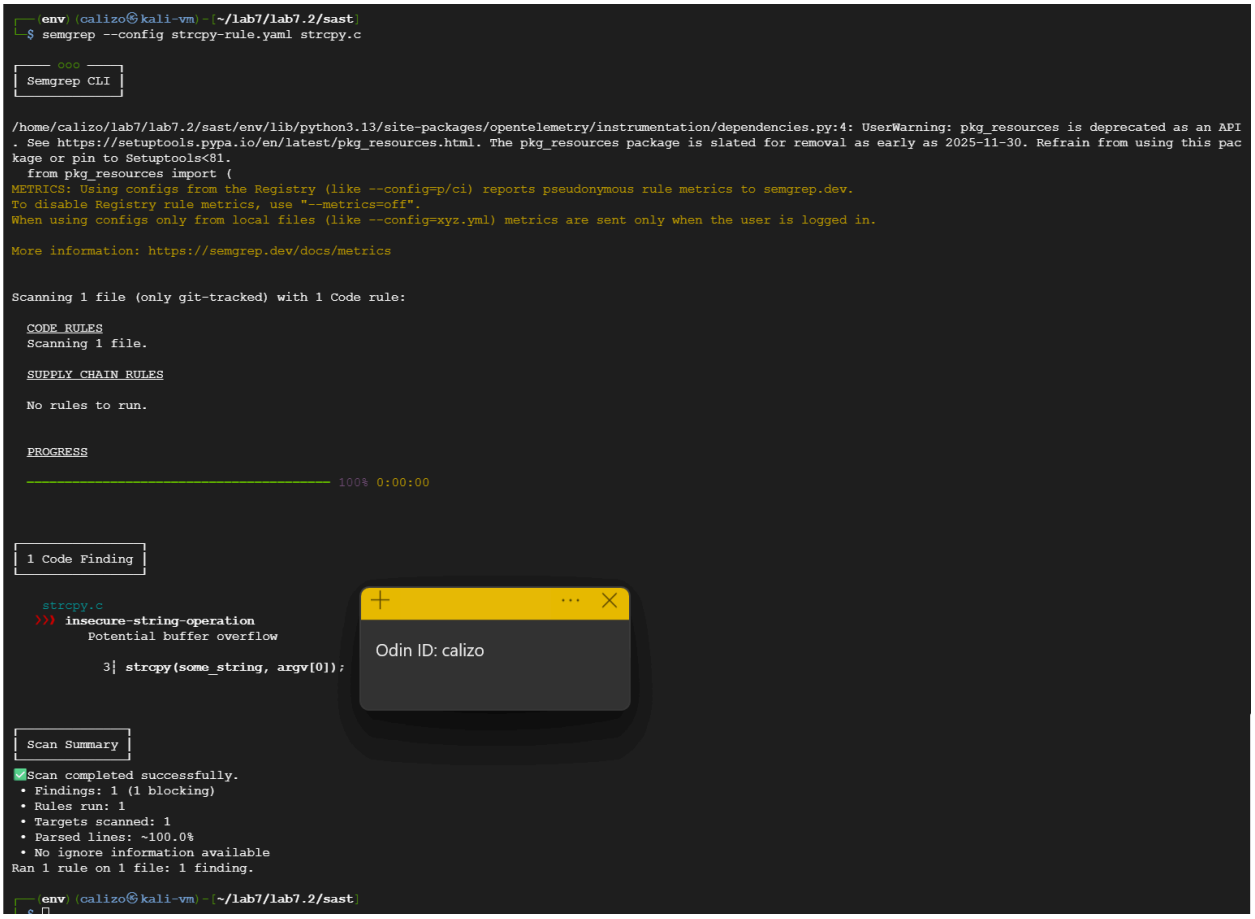
5. Take a screenshot showing successful exploitation of the binary



Lab 7.2 Static Application Security Testing

Step 2: Identifying insecure library calls

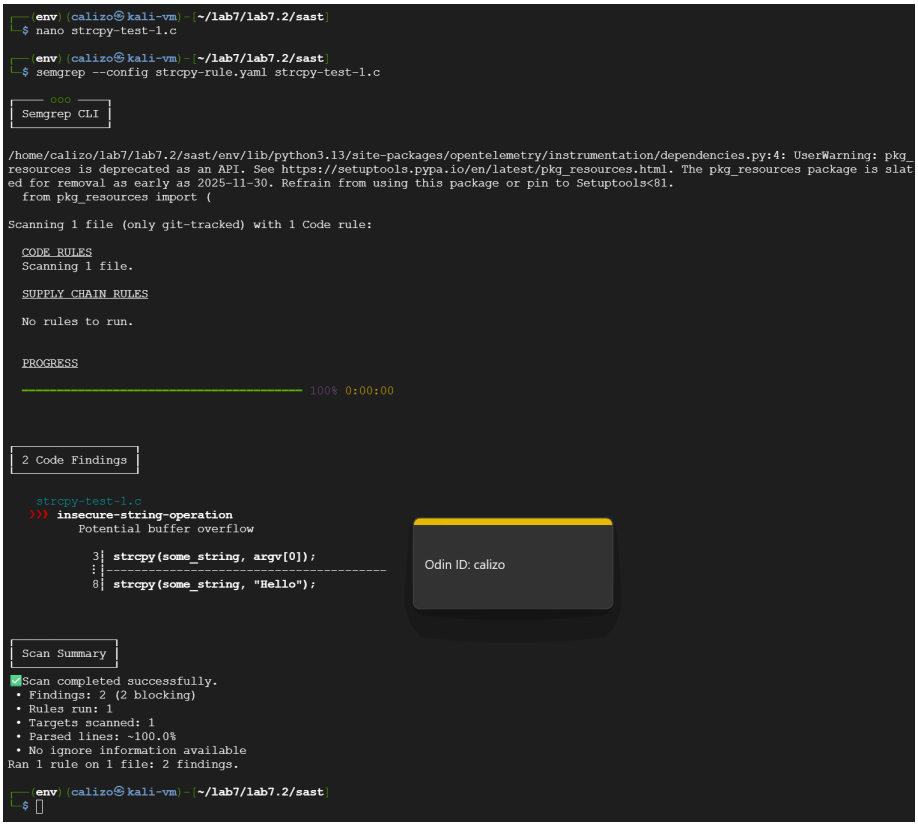
Take a screenshot showing the code finding that the rule has identified



Step 3: False Positives in Static Analysis – strcpy-test-1.c

This step highlighted one of the primary challenges in static application security testing (SAST): false positives. While tools like Semgrep are fast and language-aware, they lack full context about runtime behavior. This can lead to alerts on code that isn’t actually vulnerable, eroding developer trust in automated scanning tools.

1. Take a screenshot showing the code findings that the rule has identified



The Semgrep CLI flagged two lines:

- Line 3: strcpy(some_string, argv[0]);
- Line 8: strcpy(some_string, "Hello");

Both were matched under the rule insecure-string-operation with the message: "Potential buffer overflow"

2. Which one is a false positive?

Only the first usage is actually dangerous.

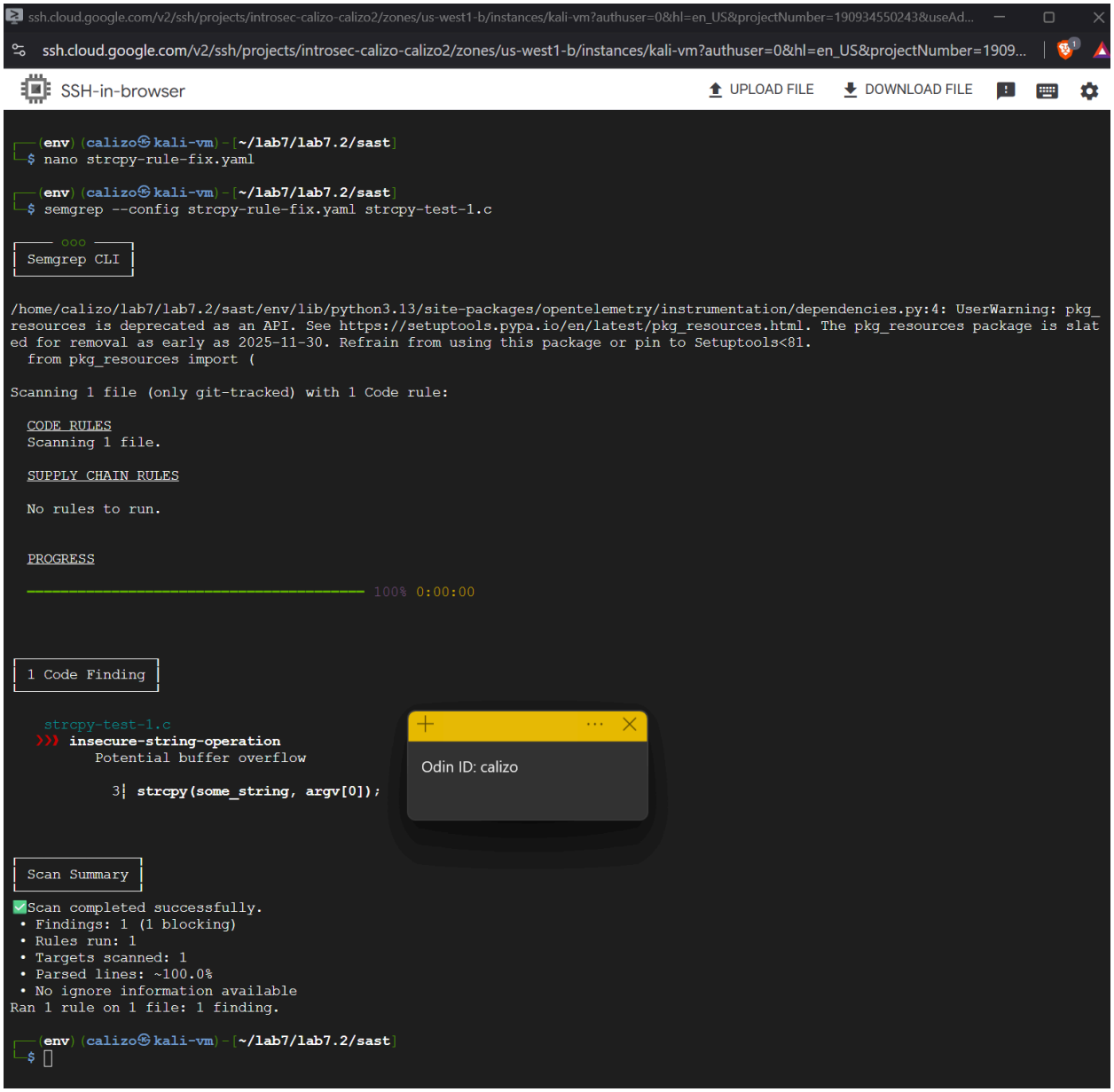
- strcpy(some_string, argv[0]) copies unchecked user input and is a real buffer overflow risk.
- strcpy(some_string, "Hello") copies a hardcoded, short string that is safely within bounds.

The alert on line 8 is a false positive. A proper rule refinement would pattern-match dynamic sources only (e.g., input, argv) rather than all strcpy() calls indiscriminately.

This step illustrates why DevSecOps teams must balance automation with precision — otherwise, developers will tune out even critical findings.

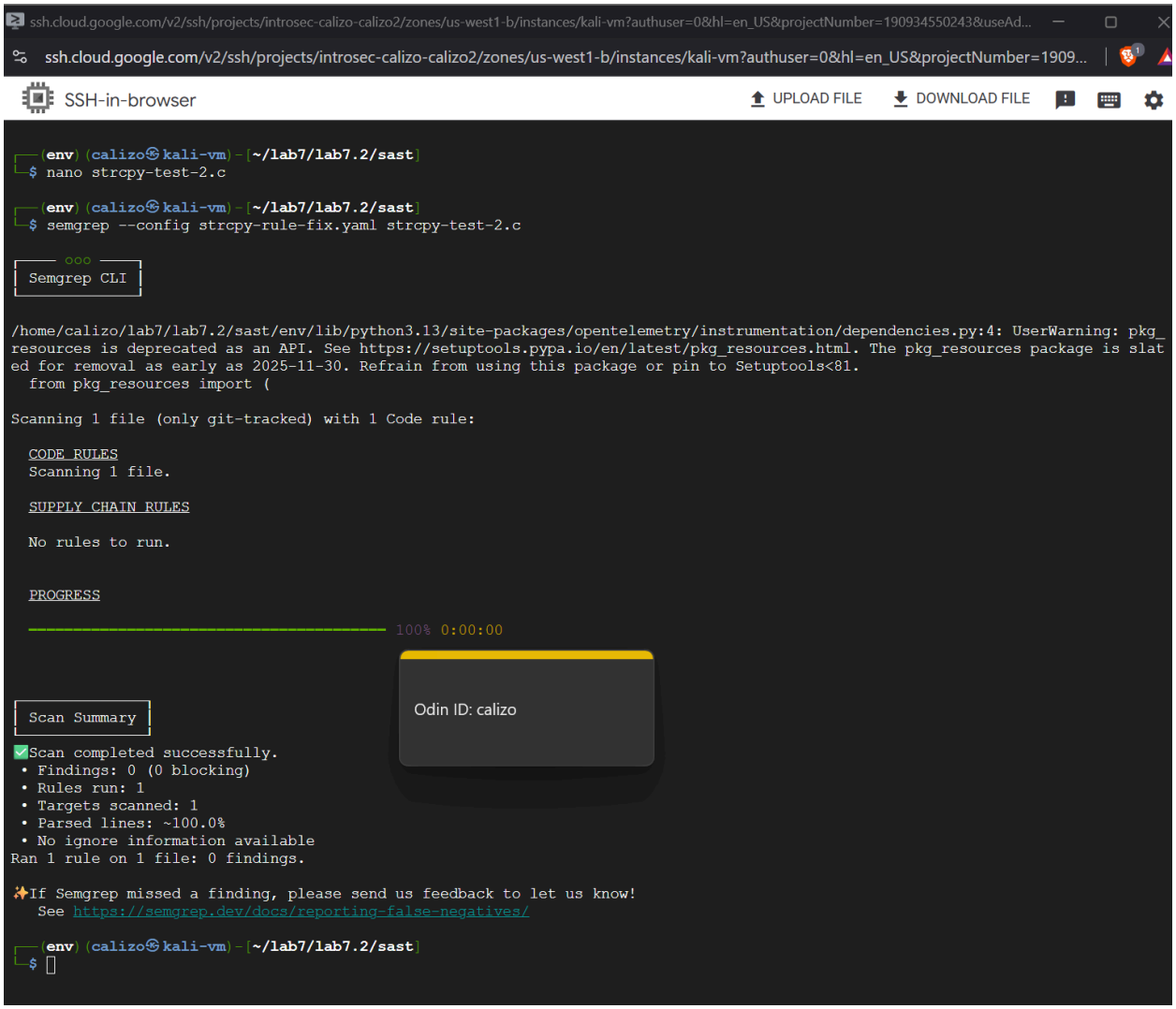
Step 4: Rewriting rules

Take a screenshot showing the false positive no longer appears



Step 5: False negatives

Take a screenshot of the result



Step 6: Rule Generation via Generative AI

1. Show the results of execution for the rule

To improve precision and reduce false positives, I prompted ChatGPT to generate a semgrep rule that flags unsafe strcpy() calls, especially those using user input or long static strings. The model produced the following rule:

```
rules:
  - id: c.unsafe.strcpy
    languages: [c]
    severity: ERROR
    message: >
      Possible buffer overflow via unsafe strcpy().
```

```
  Avoid copying user input or long strings without bounds checking.
patterns:
  - pattern: strcpy($BUF, $SRC)
  - pattern-not: strcpy($BUF, "??")
metadata:
  category: security
  cwe: "CWE-120"
```

This rule was saved as strcpy-ai-rule.yaml and executed using:
semgrep --config strcpy-ai-rule.yaml strcpy-test-1.c strcpy-test-2.c

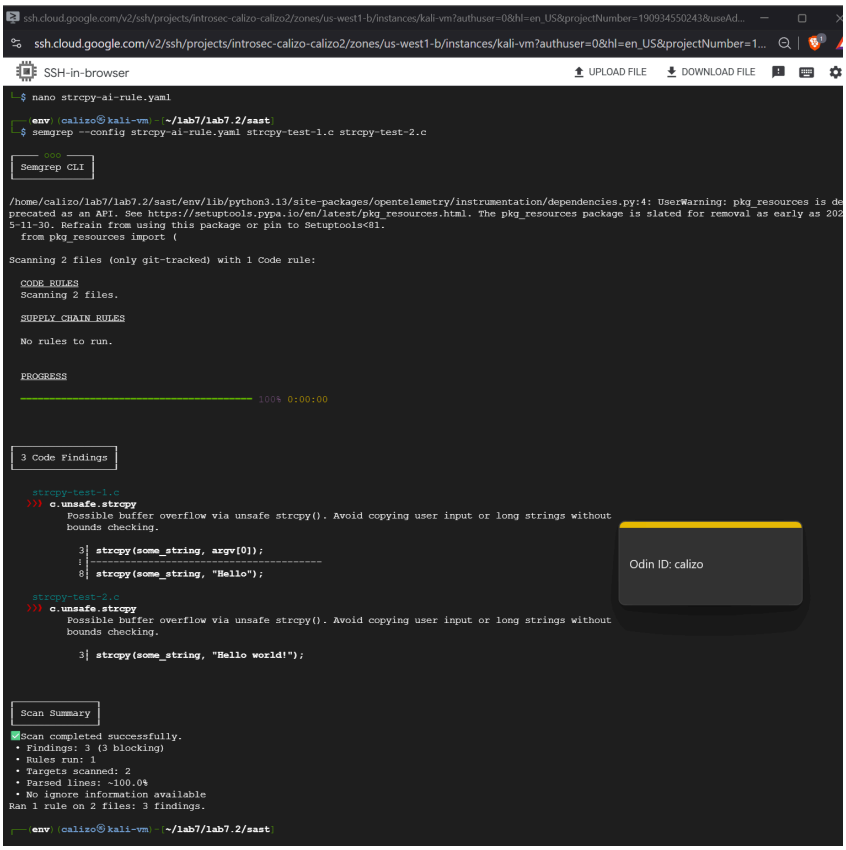
The output correctly flagged:

- strcpy(some_string, argv[0]) in str0() – user input
- strcpy(some_string, "Hello world!") in str2() – long constant

And it ignored:

- strcpy(some_string, "Hello") in str1() – a safe constant

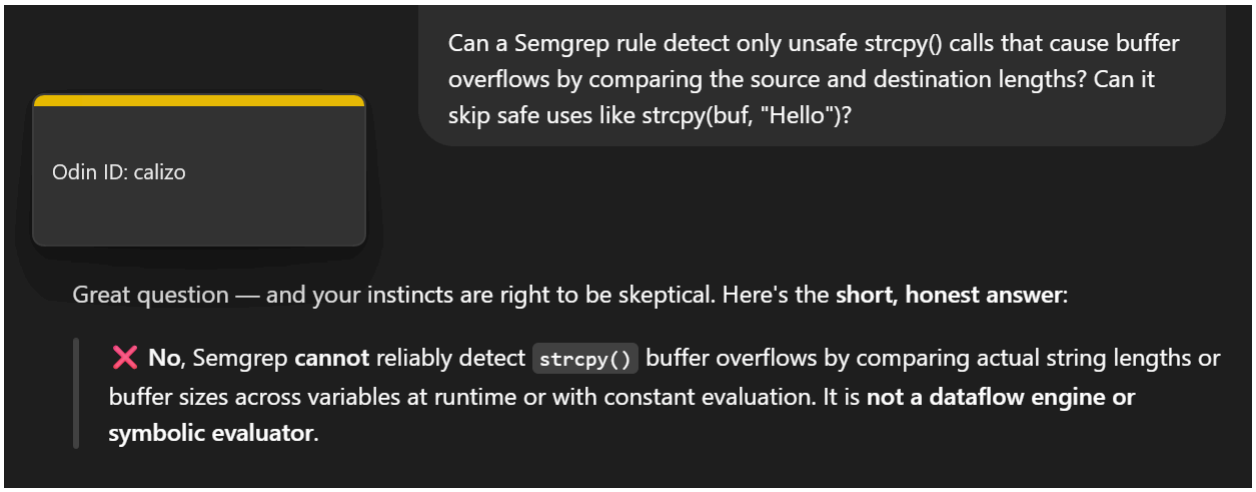
Success: The rule was effective for the provided context, validating both unsafe usage and the ability to skip obvious safe ones.



2. Ask the generative AI model whether a semgrep rule is possible for correctly returning
To be thorough, I also asked ChatGPT whether semgrep could reliably reason about buffer overflows by comparing string lengths at compile time. The model responded:

"Semgrep cannot reliably detect strcpy() buffer overflows by comparing actual string lengths or buffer sizes at runtime or with constant evaluation. It is not a dataflow engine or symbolic evaluator."

This makes sense—semgrep excels at surface-level pattern matching, not semantic or symbolic reasoning.



Semgrep can approximate detection of unsafe strcpy() usage through clever rule heuristics, but ultimately lacks the semantic depth for comprehensive detection without sacrificing either coverage or precision. Generative AI helped surface this trade-off clearly and provided a viable middle-ground rule.

Step 7: Static Testing via Generative AI

1. Show the results of each. Does the LLM produce any false positives or negatives?

Nope—no false positives or negatives were produced in my case. Here's how each evaluation turned out based on the LLM's reasoning and output:

- str0() was correctly flagged as vulnerable. The LLM gave a strong breakdown—highlighting the unbounded strcpy() usage on argv[0], which can be

arbitrarily long. It correctly labeled this as a classic stack-based buffer overflow with exploit potential (e.g., stack smashing, ROP, DoS).

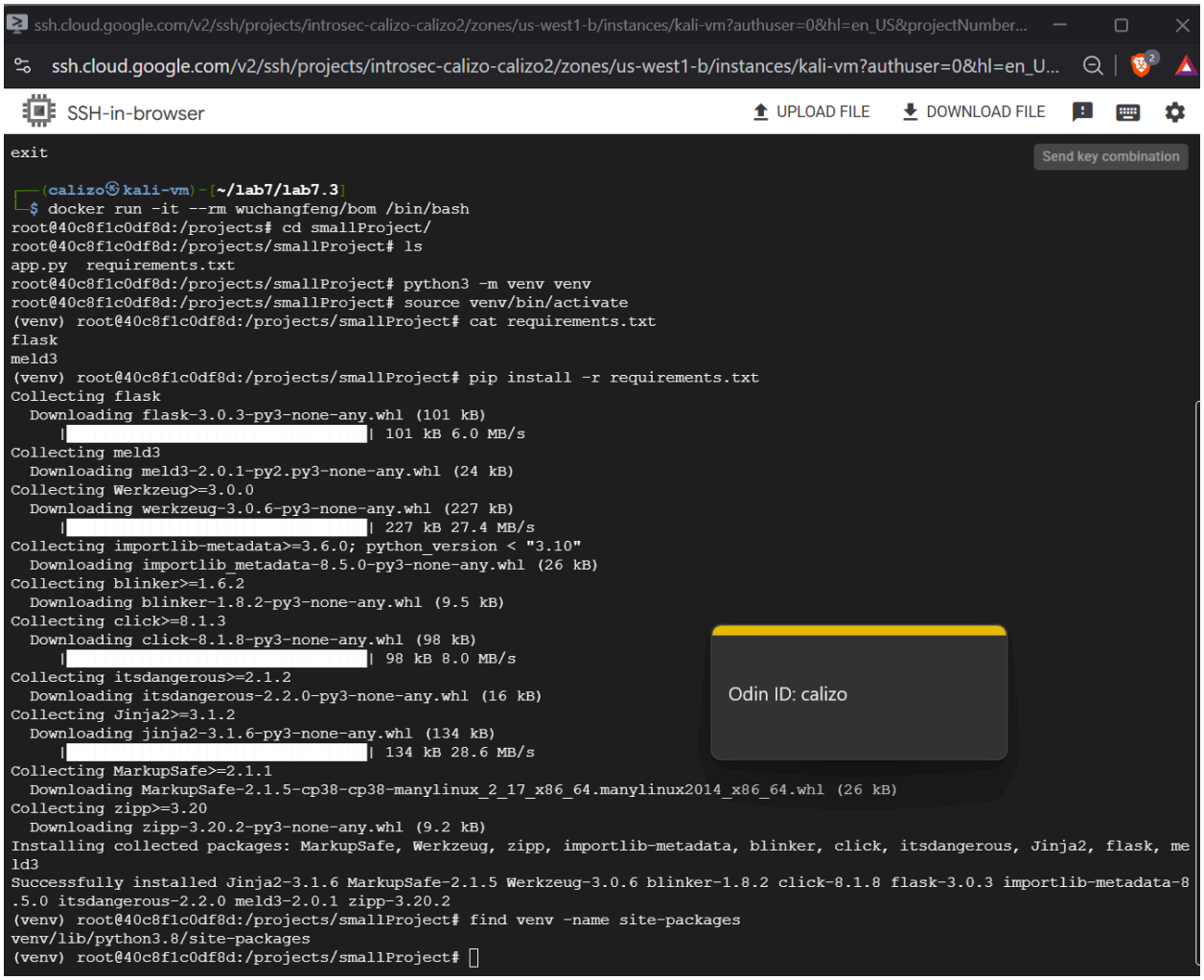
- str1() was correctly marked as not vulnerable. It reasoned that copying the compile-time constant "Hello" (6 bytes total) into a 10-byte buffer was safe and left sufficient padding. The model also emphasized that there's no user input variability here, so no risk of overflow.
- str2() was also correctly identified as vulnerable. The LLM called out that "Hello world!" is 13 bytes including the null terminator, which exceeds the 10-byte buffer. The explanation even included overflow consequences like corruption of saved frame pointers and return addresses.

So overall, the LLM gave solid, context-aware analysis—accurately flagging both overflow conditions and recognizing when code was safe. Zero false classifications in this round.

Lab 7.3 Supply-Chain Security

Step 4: Python virtual environment and packages

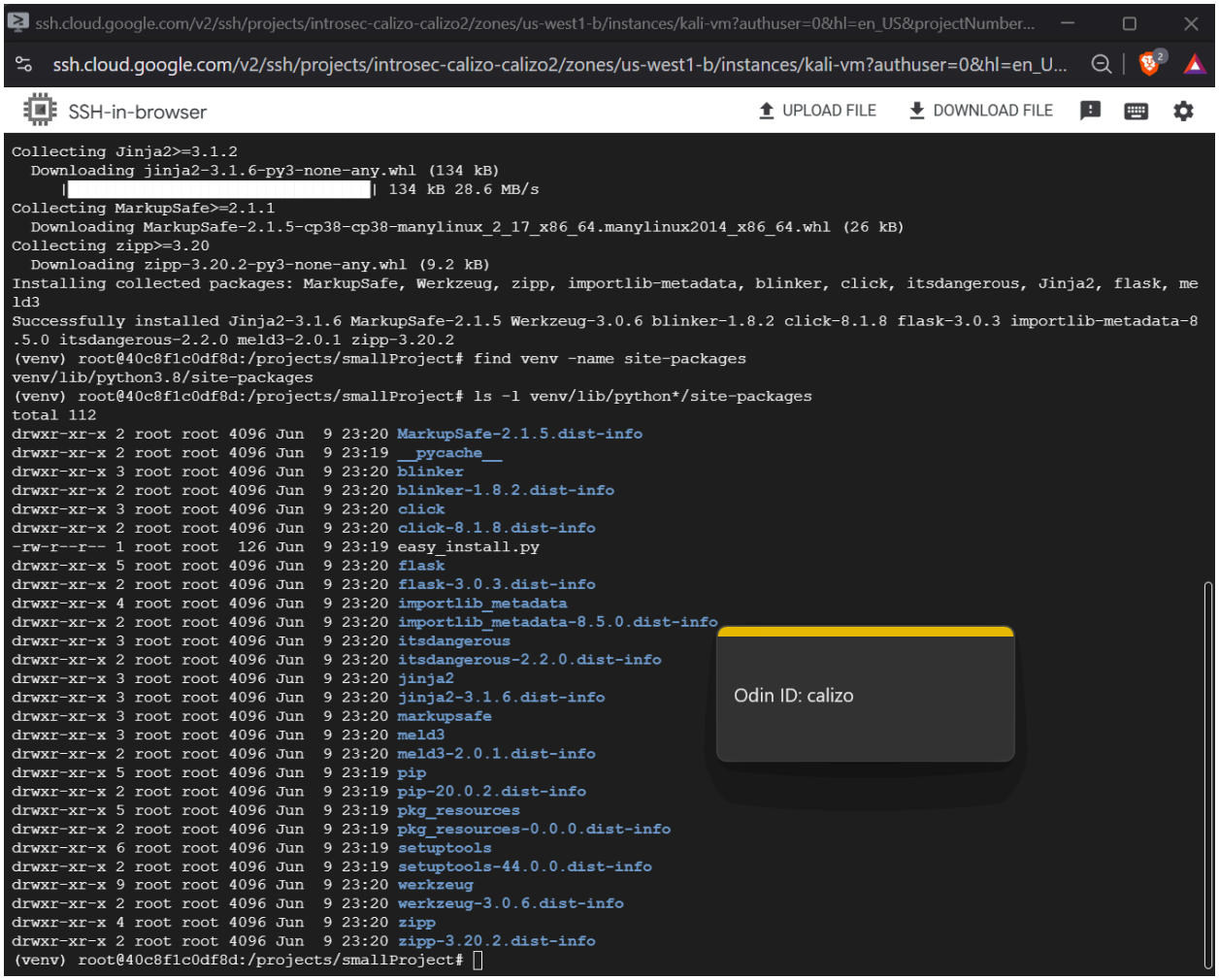
1. What directory contains all of the packages installed by pip?
The installed Python packages reside in:
venv/lib/python3.8/site-packages/



This is the standard directory structure within a virtual environment where pip installs all project-specific dependencies. This isolation ensures packages installed for this project don't interfere with system-wide Python modules or other virtualenvs.

2. Screenshot showing a listing of this directory

Shown below is a ls -l listing of the site-packages directory:



As seen above, the directory includes critical Flask stack components such as:

- flask, flask-3.0.3.dist-info
- jinja2, jinja2-3.1.6.dist-info
- werkzeug, click, itsdangerous, etc.

These subdirectories indicate that not only the top-level packages from requirements.txt were installed, but their transitive dependencies were also resolved automatically.

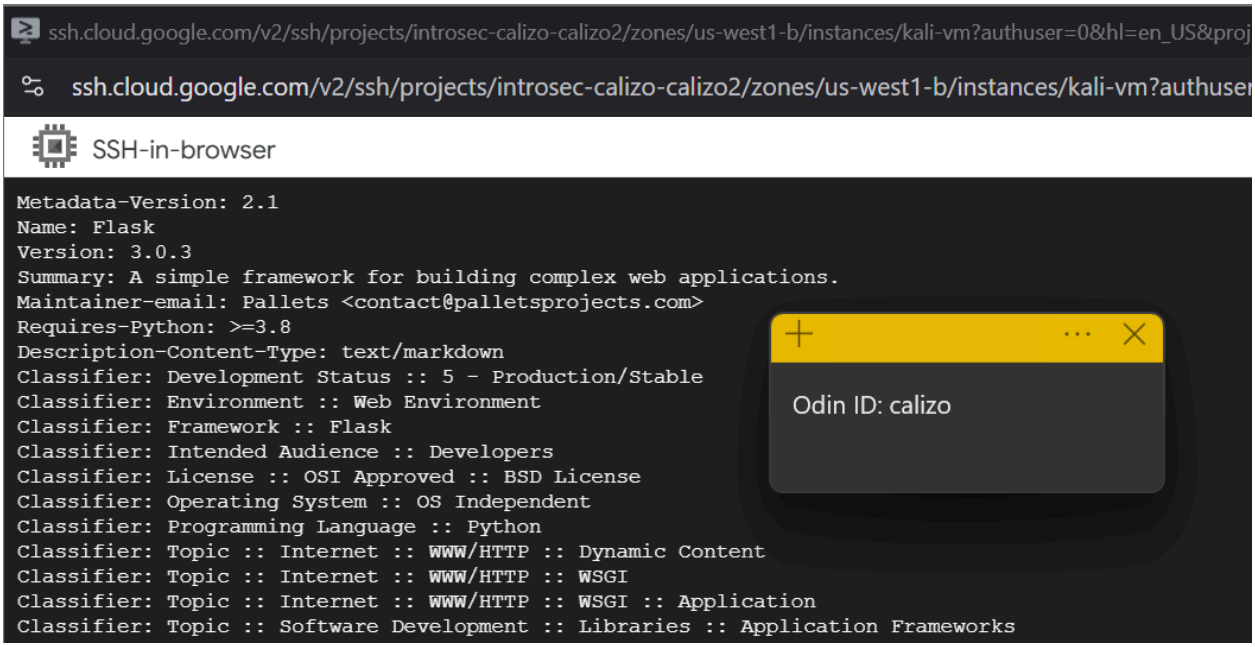
Step 5: Finding abandoned packages

1. What organization maintains the Flask package?

From inspecting the METADATA file inside the flask-3.0.3.dist-info/ directory, the Maintainer-email is listed as:

Pallets <contact@palletsprojects.com>

So the organization responsible for maintaining Flask is the Pallets project.



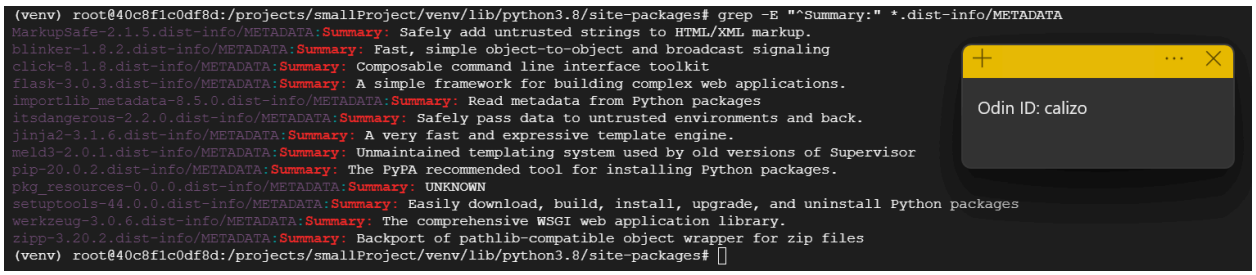
2. Which package being used is no longer being maintained?

Looking at the Summary: metadata fields across all installed packages via the command:

grep -E "^Summary:" *.dist-info/METADATA

Most packages have proper descriptions. However, one entry stands out:

pkg_resources-0.0.0.dist-info/METADATA:Summary: UNKNOWN



The pkg_resources package is labeled with UNKNOWN, which typically implies a placeholder or absence of a properly maintained summary. This is a strong signal that it's no longer being actively maintained, or at the very least, its metadata hasn't been updated to modern packaging standards. Thus, pkg_resources is the likely abandoned package here.

Step 9: Check Packages for CVEs

1. What is the Vuln ID of this CVE?

The vulnerability tied to one of the Flask-related dependencies is:
CVE-2021-33026

It targets the Flask-Caching extension (up to and including version 1.10.1) and was published on May 13, 2021. The NVD rates it with a CVSS 3.1 score of 9.8, which classifies it as Critical—typically implying unauthenticated remote code execution.

2. What is the name of the associated Weakness and its Weakness ID?
The CWE associated with CVE-2021-33026 is:

- Name: Deserialization of Untrusted Data
- CWE ID: CWE-502

This kind of vulnerability is especially dangerous because it occurs when attacker-controlled data is deserialized without validation, potentially leading to arbitrary code execution. It's a key reason why many modern web applications are shifting toward safer data formats like JSON, which doesn't support complex object graphs or executable payloads.

Step 11: Snyk

1. What are the CVE numbers for the critical vulnerabilities this project has?

The following critical vulnerabilities were detected by Snyk during the scan of SUPER_MASSIVE_PROJECT:

- SNYK-PYTHON-CERTIFI-5805047 / CVE-2023-37920
  (https://www.cve.org/CVERecord?id=CVE-2023-37920)
  - Improper Following of a Certificate's Chain of Trust in certifi@2021.5.30

- SNYK-PYTHON-PILLOW-2331901 / CVE-2022-22817
  (https://www.cve.org/CVERecord?id=CVE-2022-22817)
  - Arbitrary Code Execution in pillow@8.3.1

- SNYK-PYTHON-PILLOW-5918878 / CVE-2023-4863
  (https://www.cve.org/CVERecord?id=CVE-2023-4863)
  - Heap-based Buffer Overflow in pillow@8.3.1

- SNYK-PYTHON-REQUESTS-72435 / CVE-2018-18074
  (https://www.cve.org/CVERecord?id=CVE-2018-18074)
  - Information Exposure in requests@2.18.0

Each of these reflects an exploitable flaw in a commonly used dependency, with direct risk implications.

2. What impact would leaving these vulnerabilities unpatched have?

Failing to patch these opens the door to serious attack vectors:

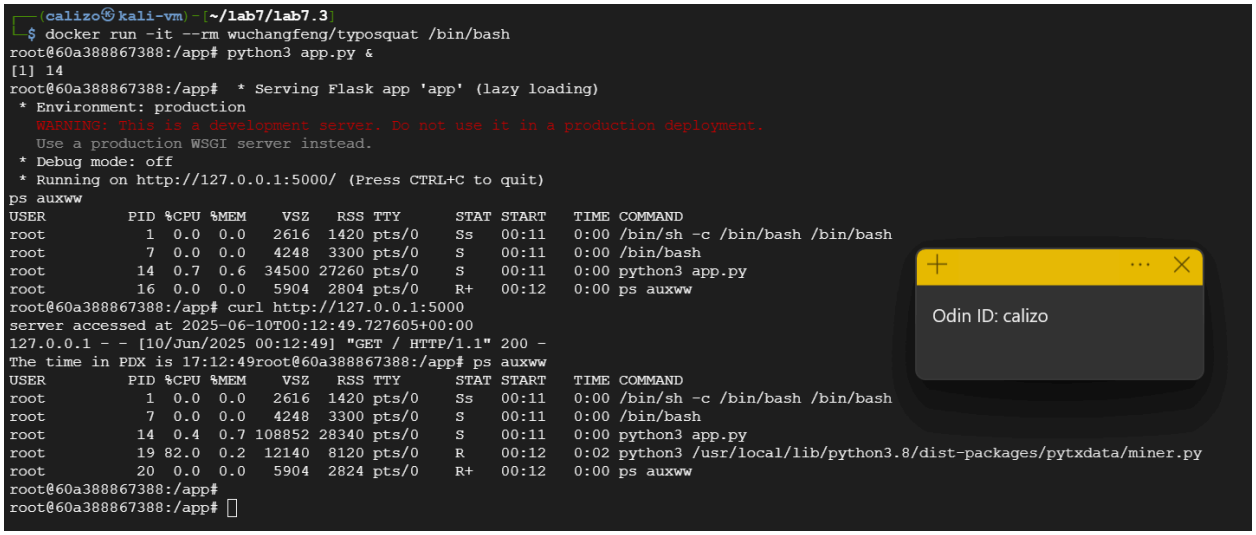
- certifi’s trust chain flaw allows malicious actors to spoof certificates, enabling man-in-the-middle attacks across supposedly secure connections.
- pillow's vulnerabilities are especially concerning: buffer overflows and arbitrary code execution make remote takeover or system compromise very realistic in image-heavy apps.
- requests’ information exposure may leak sensitive data—think API tokens, credentials, or session identifiers—making it ripe for exfiltration or lateral movement within a compromised system.

Together, these could lead to:

- Remote Code Execution (RCE)
- Compromised data integrity and confidentiality
- Supply chain poisoning
- Full application takedowns or privilege escalations

In short: unpatched, these aren’t just bugs—they're breach invitations.

Step 13: Container Setup



1. What is the PID of the new process and the command line used to launch it?

After running curl http://127.0.0.1:5000, I reissued ps auxww and observed a new child process spawned by the Flask web app.

- PID: 19
- Command: python3 /usr/local/lib/python3.8/dist-packages/pytxdata/miner.py

This is clearly sketchy — the use of miner.py in a module named pytxdata is highly suspect and not part of any known standard package.

2. What %CPU is the process consuming?

- %CPU: 82.0 — Extremely high.

This is a red flag for cryptomining behavior. CPU-hogging, background-spawned, stealth processes like this often reflect crypto-related operations, especially if tied to obscure modules.

3. What is the name of the Python package which contains the code this process uses?

- Suspicious package: pytxdata

This is almost certainly a typosquatted package — likely mimicking something legitimate like pytz or pandas, but the name and behavior (launching miner.py) expose its malicious intent.

This exercise reinforced how vulnerable codebases can become from something as seemingly trivial as a misspelled package. Typosquatting combined with dynamic payload behavior is a real threat, and catching this with live tracing and inspection adds strong evidence for securing our software supply chains.

Step 14: Investigating Packages

1. From the Summary field, what does this package purport to provide?

The Summary field in the METADATA file claims that the package provides "The Olson timezone database for Python." This matches the original intent of the legitimate pytzdata package, reinforcing the typosquat's deceptive authenticity.

2. What is the URL of the original repository that the code has been forked from?

The direct_url.json file shows the package was pulled from:
https://github.com/typocft/pytxdata.git
However, visiting this repo confirms it's a malicious fork.

The original upstream repository (seen on GitHub from UI inspection) is:

- https://github.com/sdispater/pytzdata

3. How does this URL differ from the URL in the Home-page field?
The Home-page field in METADATA also claims the repo is:
https://github.com/sdispater/pytxdata — which is deceptively one letter off from the legitimate pytzdata.
So while the "home-page" field tries to impersonate the original repo with a subtle spelling difference, the actual code was pulled from an entirely separate repo (typocft) with malicious intentions.

Lab 7.4 Tor and the Dark Web

Step 3: Comparing Accesses via Tor and Clear Web

1. Take a screenshot showing the IP addresses and User-Agent fields for each and note the differences.



Attached screenshots show the following:

- Chrome web browser (left):
  - IP Address: 73.240.180.242
  - User-Agent: Chrome-based string (AppleWebKit/537.36, Chrome/137.0.0.0)

- Tor browser (right):
  - IP Address: 2a0b:f4c2::31 (IPv6 exit node)
  - User-Agent: Firefox-based string (Gecko/20100101, Firefox/128.0)

Key Differences:

- The IP addresses differ substantially: my real IP (Comcast, Oregon) vs. the anonymized Tor exit node (IPv6 address).

- The User-Agent reflects different browsers: Chrome on the normal browser and Firefox on Tor (expected, since Tor Browser is Firefox-based).

2. Provide an explanation of what these values might be from and whether it is a cause for concern.



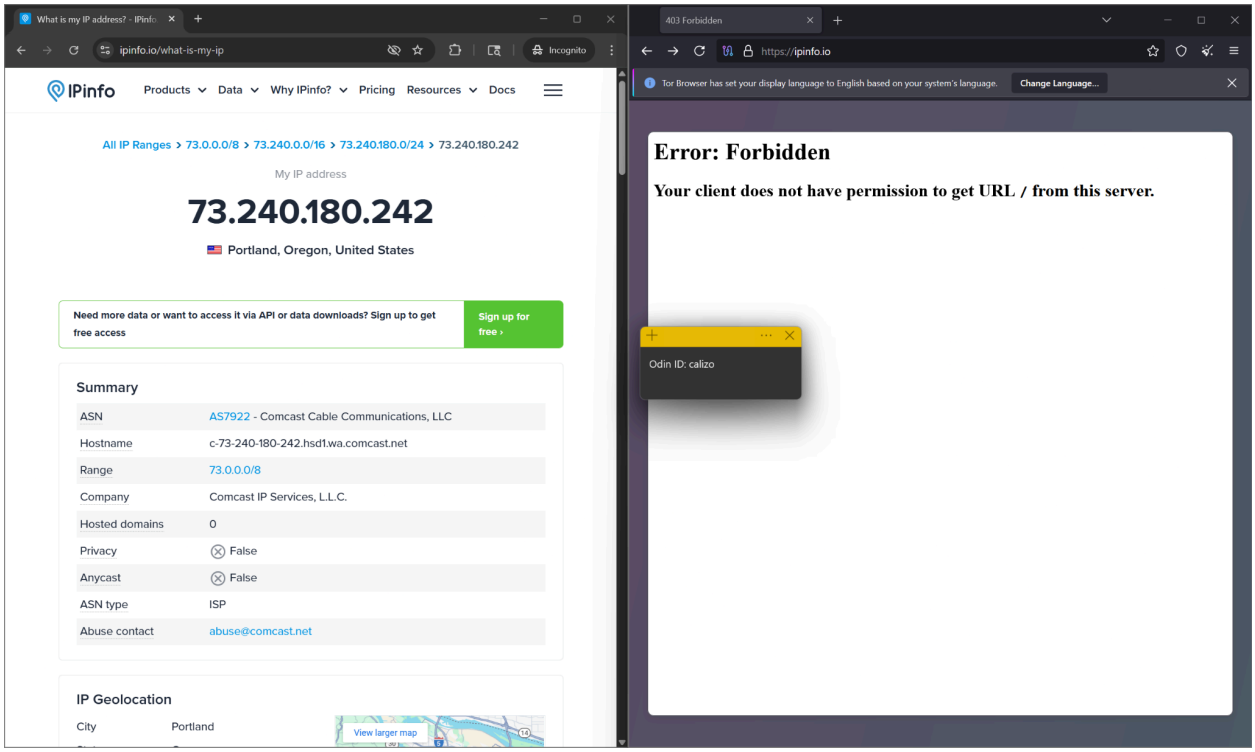
The X-Forwarded-For header in each response provides insight into what intermediate proxies or relays may have passed along the request:

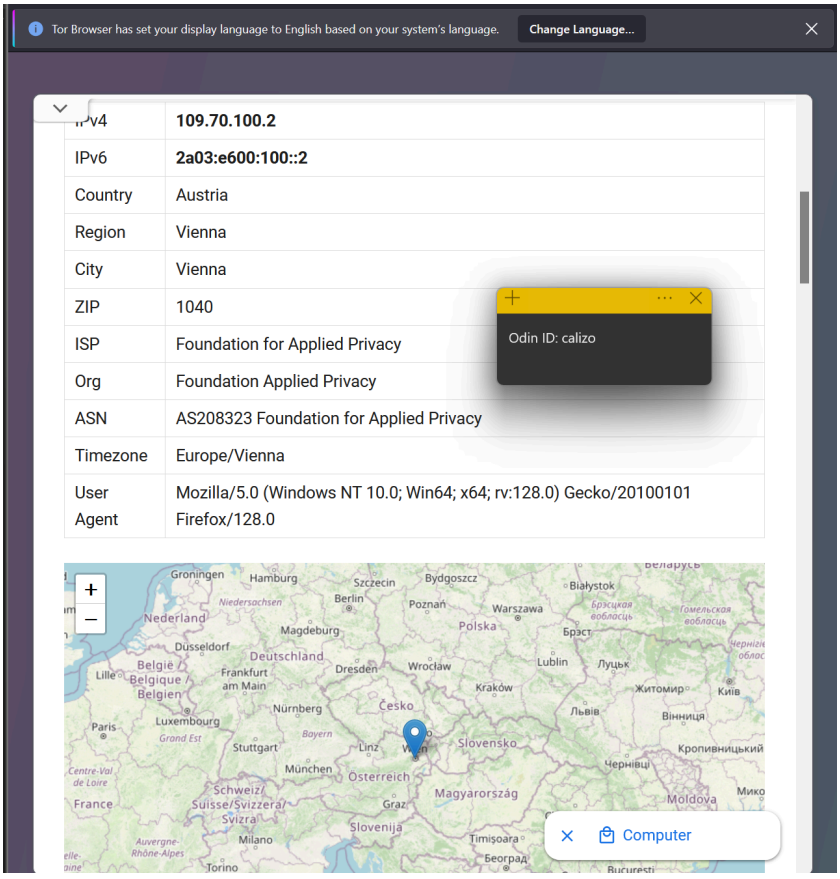
- Clear web: Shows 73.240.180.242,34.160.111.145 — the first is my IP, while the second may be an upstream proxy/load balancer.
- Tor browser: Shows 2a0b:f4c2::31,2600:1901:0:b:2bd:: — both are Tor exit relay IPv6 addresses.

A reverse DNS (nslookup) confirmed that 2a0b:f4c2::31 resolves to a Tor exit node in Berlin, hosted by the artikel10.org project. This is expected behavior — the X-Forwarded-For header is not a vulnerability here, but it does reveal some metadata that could be used by web services for rudimentary fingerprinting or blocking.

It's not a critical concern unless the destination server logs and aggregates this data — but in anonymous browsing contexts, every leaked field matters.

3. Take a screenshot showing the IP addresses returned and explain any differences between these IP addresses and the ones returned by https://ifconfig.me





- Screenshots confirm that:
    - Chrome browser via ipinfo.io shows 73.240.180.242 (Portland, Oregon)

- Tor browser on ipinfo.io fails with a 403 Forbidden (likely due to IP blocks on Tor exit nodes)

As a workaround, I used whatsmyip.org on the Tor browser, which returned:

- 109.70.100.2 (IPv4)
- Located in Vienna, Austria (Foundation for Applied Privacy)

This difference exists because Tor routes traffic through randomly selected exit nodes around the world — while ifconfig.me used one exit relay, the site I used later resolved through a different one, likely due to circuit rotation.

4. What is the geographic location of the IP address returned when accessing the site from the Tor browser?

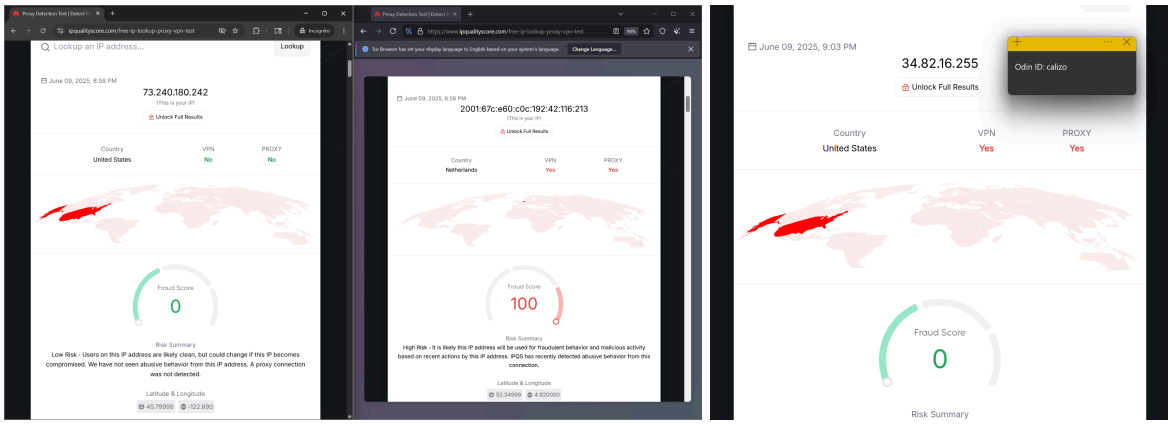
Vienna, Austria

- ASN: Foundation for Applied Privacy (AS208323)
- Exit node IP: 109.70.100.2

Based on the reverse DNS and IP lookup, this was the active Tor exit relay at the time of access.

Step 4: Threat Intelligence

1. Show the fraud scores of each. Explain why you believe the results are either accurate or inaccurate.

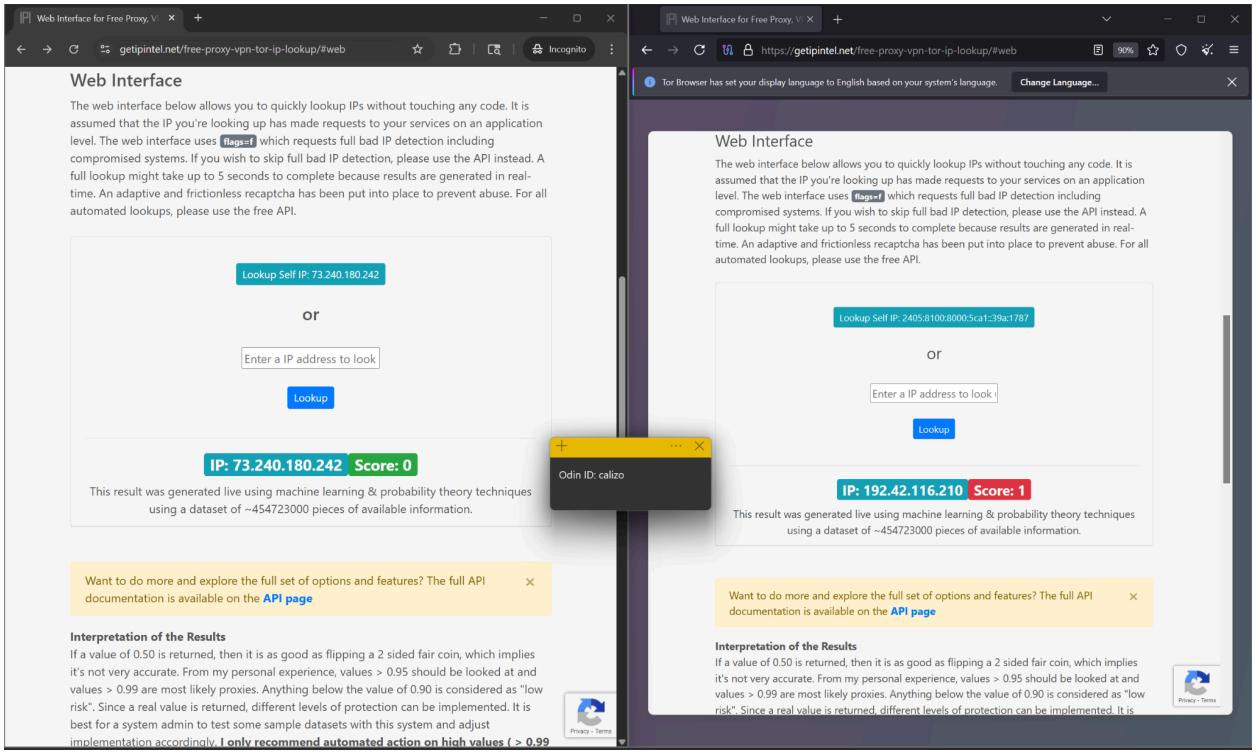


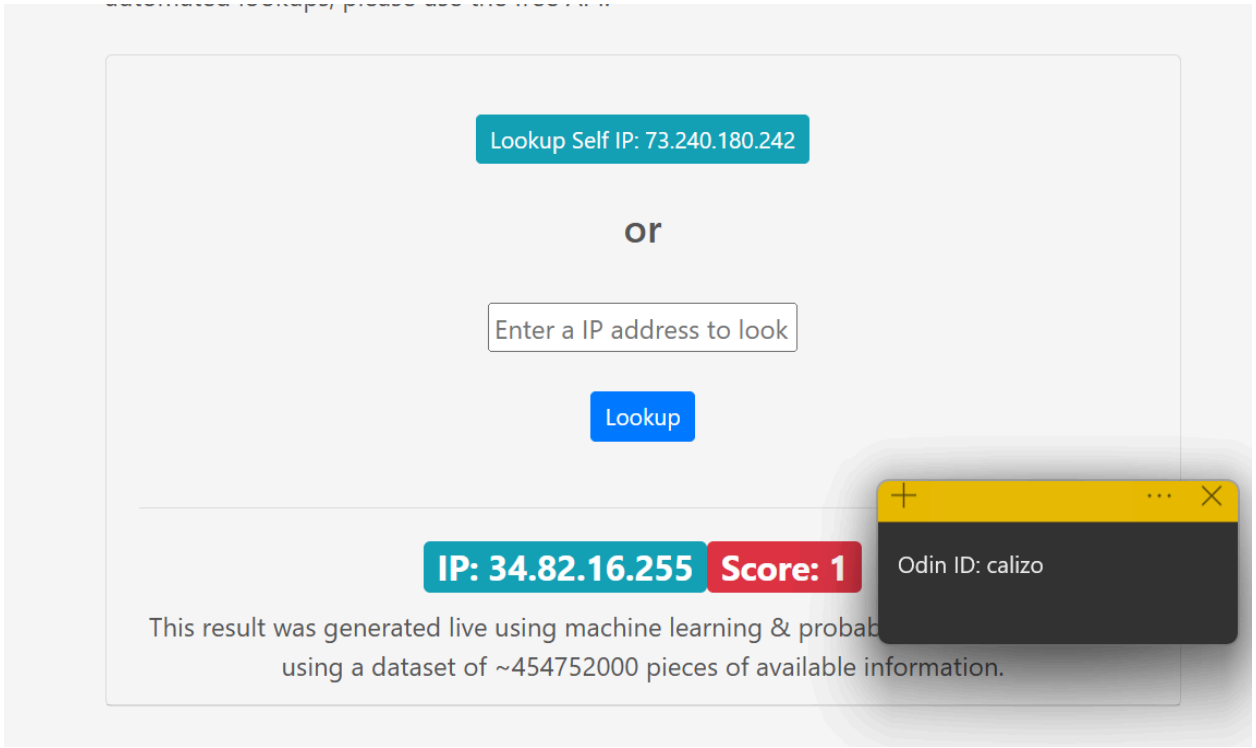
IPQualityScore Results:

- My personal IP (73.240.180.242)
  - Fraud Score: 0
  - Country: United States
  - VPN/Proxy: No
  - Accurate – This IP is tied to my residential ISP (Comcast) and isn't behind a proxy or VPN, so the low risk and fraud score makes sense.

- Tor IP (2001:67c:e60:c0c:192:42:116:213)
    - Fraud Score: 100
    - Country: Netherlands
    - VPN/Proxy: Yes
    - Accurate – Tor exit nodes are widely flagged as proxies and known for anonymity, so this IP being flagged with a perfect fraud score is expected.
- oregonctf.org IP (34.82.16.255)
    - Fraud Score: 0
    - Country: United States
    - VPN/Proxy: Yes
    - Mixed Accuracy – The fraud score being 0 is accurate from a "legit server" standpoint, but it is hosted on GCP so the VPN/Proxy flag makes sense since many services flag cloud providers as anonymizing infrastructure.

2. Show the scores of each address. Are they consistent with that of IPQualityScore? Explain why they might differ.





GetIPIntel Results:

- My personal IP (73.240.180.242)
    - Score: 0

    - Consistent – Matches IPQualityScore. Personal residential IP, not suspicious.
- Tor IP (192.42.116.210, one of the detected exit nodes)
    - Score: 1
    - Consistent – High risk, flagged with certainty as a Tor node or anonymizing service.
- oregonctf.org IP (34.82.16.255)
    - Score: 1
    - Slightly Inconsistent – While IPQualityScore flagged it as VPN/Proxy but gave it a fraud score of 0, GetIPIntel gave it a full 1. That's likely due to its data center origin (Google Cloud), which is often flagged by GetIPIntel regardless of actual abuse.

The discrepancy in the oregonctf.org results highlights a difference in scoring philosophy—IPQualityScore emphasizes abuse behavior (none here), whereas GetIPIntel weighs infrastructure type (GCP = possibly suspicious). Combining both tools gives a fuller picture in real-world threat intelligence workflows.

Step 5: Accessing Onion Services

1. What is the Bitcoin address of the person who runs the service?
After combing through the full content of the onion service at danielas3rtn54uwmofdo3x2bsdifr47huasnmbgqzfrec5ubupvtpid.onion, I was unable to locate any Bitcoin address. Either the BTC field was removed from the current version of the site or the lab instructions are referencing outdated content. I searched thoroughly through all headers, footers, FAQ sections, and mail setup panels — not a single crypto wallet address was listed.

2. What is the purpose of this site?
The second .onion URL led to SecureDrop, a platform designed to facilitate secure and anonymous document sharing between sources (e.g., whistleblowers) and journalists or NGOs. The system is fully open source, emphasizes privacy, and is available in 22 languages. The homepage stresses that organizations can install SecureDrop on their infrastructure to receive confidential tips, source material, or leads in a way that hides both sender and recipient metadata — classic end-to-end anonymity use case for Tor.

Step 9: Visit the Onion Site

